G H Raisoni College of Engineering & Management
An Empowered Autonomous Institute, Affiliated to Savitribai Phule Pune University
Approved by AICTE, New Delhi and Recognized by Govt. of Maharashtra, NAAC Accredited "A+" Grade
Navin Gat No.1200, Domkhel Road, Wagholi, Pune - 412 207 (India)
T: +91 - 9604787185/186 | E: ghrcem.pune@raisoni.net | W: www.ghrcemp.raisoni.net

Department of Information Technology

SY. B.Tech 2023 Scheme (As per NEP 2020)

Semester- III

UNIT 1.

Introduction To Data Structures And Algorithms

Mapping of Course Outcomes- CO1

CO1: DISCUSS BASICS OF DATA STRUCTURES AND ANALYZE COMPLEXITY OF VARIOUS ALGORITHMS

Prepared by **Prof. Yogesh Gite**

Topics

Introduction:

- Problem Solving –
  - From Problem to Program - Problem, Solution, Algorithm, Data Structure & Program
- Types of Data Structures
  - Linear & Nonlinear , Static & Dynamic, Persistent & Ephemeral data structures
- Concept of ADT
- Operations on Data Structures.

Arrays:

- Arrays as ADT
- Representation of Linear array in Memory
- Traversing Linear Arrays
- Inserting and deleting.

Topics (...continued)

Algorithms:

- Introduction to algorithm,
- Characteristics of algorithm,
- Pseudo-code and flowchart,
- Space complexity, time complexity,
- Order of growth,
- Asymptotic notation- big-o, theta and omega,
- Finding complexity using step count method.

Introduction

- From Problem to Program
  - Problem
  - Solution
  - Algorithm
  - Data Structure
  - Program

Introduction

From Problem to Program

- **Problem** is a **task** to be performed. It is best thought of in terms of **inputs** and **matching outputs.**

  Problem definition should **not include** any **constraints** on ***how*** the problem is to be solved.

- **Solution** method should be developed only after the problem is precisely defined and thoroughly understood.

  include any **constraints** on ***how*** the problem is to be solved

- **Algorithm** is a **method** or a **process** followed to solve a problem.

Introduction

From Problem to Program

- **Program** is a list of **instructions** or program **statements** composed in such a way as to **enable** a computer to solve a problem.
- Problem is broken down into successively smaller parts.
- These **parts** should **form a well defined structure,** the large complex problem at the top the small easy to handle problems at the bottom. Hence the term top down programming.
  - State the problem in English
  - Examine the problem and break down into several parts.
  - Examine the parts and refine into smaller parts.
  - Sketch a picture/structure plan
  - Write the main program with references to the subprograms.
  - Test the program.

Data Structures

- **Data** is collection of numbers, characters and symbols which is combined to represent **Information.**
- Data structure is **a storage that is used to store and organize data**. Also used for processing, retrieving, and storing data.
- It is a **way of arranging** data on a computer so that it can be **accessed** and **updated** efficiently.
- Data structure is a **representation** of data and the **operations** allowed on that data.
- Data Structure are the method of representing of **logical relationships** between individual data elements related to the **solution** of a given problem
- Data structures are a crucial part of **designing efficient software** and play a critical role in **algorithm design** and how those algorithms are used within computer programs

Need of Data Structure

Why are data structures important?

- Typical base data types, integers or floating-point values, are generally **insufficient** to capture the logical goal for data processing and use.
- Applications that consume, manipulate and produce information must understand **how data** should be **organized** to simplify processing.
- Data structures bring together the data elements in a logical way and facilitate the effective use, persistence and sharing of data.
- They provide a formal model that describes the way the data elements are organized.
- Data structures are the building blocks for more sophisticated applications.
- They are designed by composing data elements into a logical unit representing an abstract data type that has relevance to the algorithm or application.

Need of Data Structure

- The structure of the data and the synthesis of the algorithm are relative to each other.
- Data presentation must be easy to understand so the developer, as well as the user, can make an efficient implementation of the operation.
- Data structures provide an easy way of organizing, retrieving, managing, and storing data.

Here is a list of the needs for data.

- Data structure modification is easy.
- It requires less time.
- Save storage memory space.
- Data representation is easy.
- Easy access to the large database

1. Stores huge data
2. Stores data in systematic way
3. Retains logical relationship
4. Provides various structure
5. Static and dynamic formats
6. Better algorithms

Data Structures – How to Choose the Appropriate

- It is not only important to use data structures, but it is also important to choose the proper data structure for each task.
- Choosing an ill-suited data structure could result in slow runtimes or unresponsive code.
- Five factors to consider when picking a data structure :
  1. What kind of information will be stored?
  2. How will that information be used?
  3. Where should data persist, or be kept, after it is created?
  4. What is the best way to organize the data?
  5. What aspects of memory and storage reservation management should be considered?

Data Structures – How to Choose the Appropriate

How to choose a data structure

- Choice of particular data model depends on two consideration:
  - It must be rich enough in structure to represent the relationship between data elements
  - The structure should be simple enough that one can effectively process the data when necessary

Data Structures – How to Choose the Appropriate

How to choose a data structure

- When choosing a data structure for a program or application, developers should consider the answers to the following three questions:
  - **Supported operations.** What functions and operations does the program need?
  - **Computational complexity.** What level of computational performance is tolerable? For speed, a data structure whose operations execute in time linear to the number of items managed -- using Big O Notation: O(n) -- will be faster than a data structure whose operations execute in time proportional to the square of the number of items managed -- $O(n^2)$.
  - **Programming elegance.** Are the organization of the data structure and its functional interface easy to use?

Data Structures – Applications

- **Storing data.** Data structures are used for efficient data persistence, such as specifying the collection of attributes and corresponding structures used to store records in a database management system.
- **Managing resources and services.** Core operating system (OS) resources and services are enabled through the use of data structures such as linked lists for memory allocation, file directory management and file structure trees, as well as process scheduling queues.
- **Data exchange.** Data structures define the organization of information shared between applications, such as TCP/IP packets.
- **Indexing**. Even more sophisticated data structures such as B-trees are used to index objects, such as those stored in a database.

Data Structures – Applications

- **Ordering and sorting.** Data structures such as binary search trees -- also known as an ordered or sorted binary tree -- provide efficient methods of sorting objects, such as character strings used as tags. With data structures such as priority queues, programmers can manage items organized according to a specific priority.
- **Searching.** Indexes created using binary search trees, B-trees or hash tables speed the ability to find a specific sought-after item.
- **Scalability.** Big data applications use data structures for allocating and managing data storage across distributed storage locations, ensuring scalability and performance. Certain big data programming environments -- such as **Apache Spark** -- provide data structures that mirror the underlying structure of database records to simplify querying.

Data Structures - Characteristics

- Data structures are often classified by their characteristics.
- The following three characteristics:
  - **Linear or non-linear.** This characteristic describes whether the data items are arranged in sequential order, such as with an array, or in an unordered sequence, such as with a graph.
  - **Homogeneous or heterogeneous.** This characteristic describes whether all data items in a given repository are of the same type. One example is a collection of elements in an array, or of various types, such as an abstract data type defined as a structure in C, a class specification in C++/ Java.
  - **Static or dynamic.** This characteristic describes how the data structures are compiled. Static data structures have fixed sizes, structures and memory locations at compile time. Dynamic data structures have sizes, structures and memory locations that can shrink or expand, depending on the use

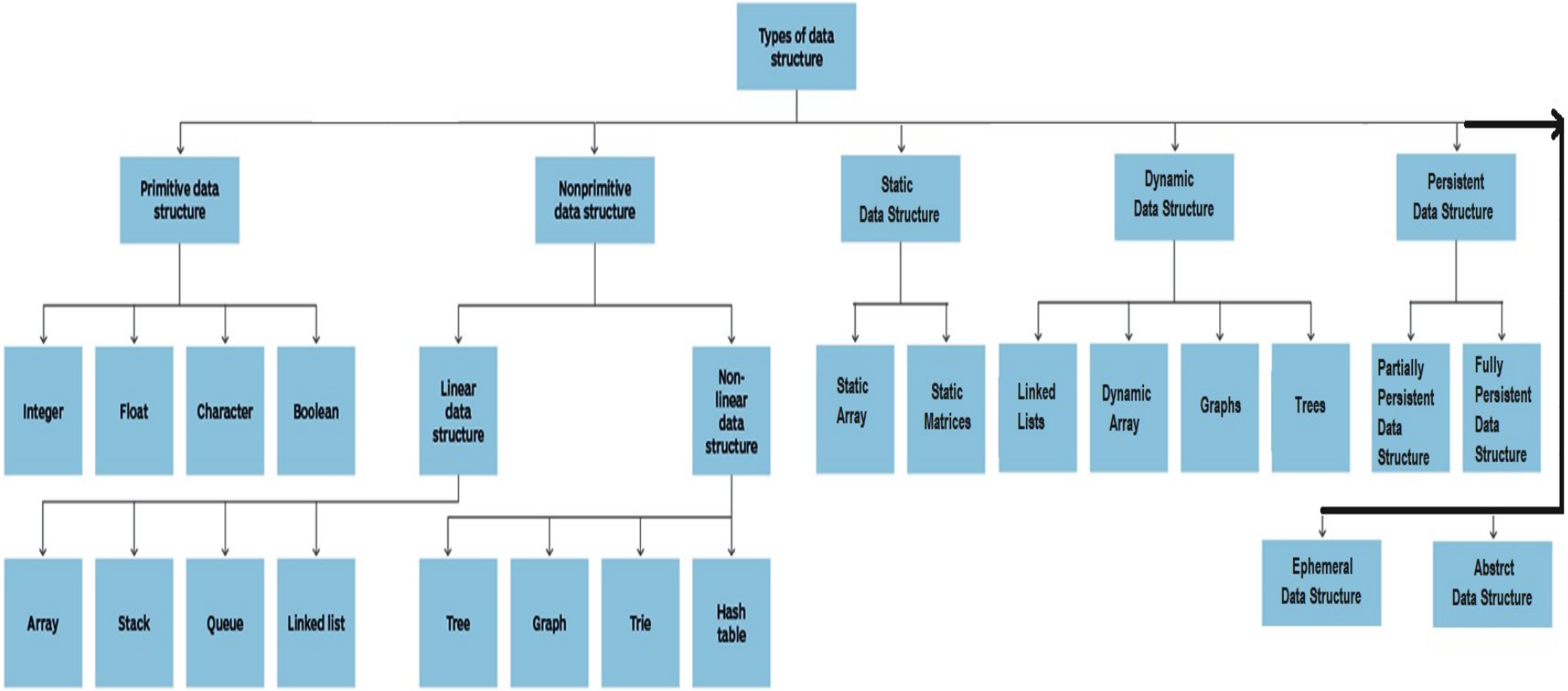
Data Structures Hierarchy

Types of Data Structures

- Several types of data structures can used while coding or solving a problem.
- The seven types of data structure are :
  - Linear Data Structure
  - Non-Linear Data Structure
  - Static Data Structure
  - Dynamic Data Structure
  - Persistent Data Structure
  - Ephemeral Data Structure
  - Abstract Data Type

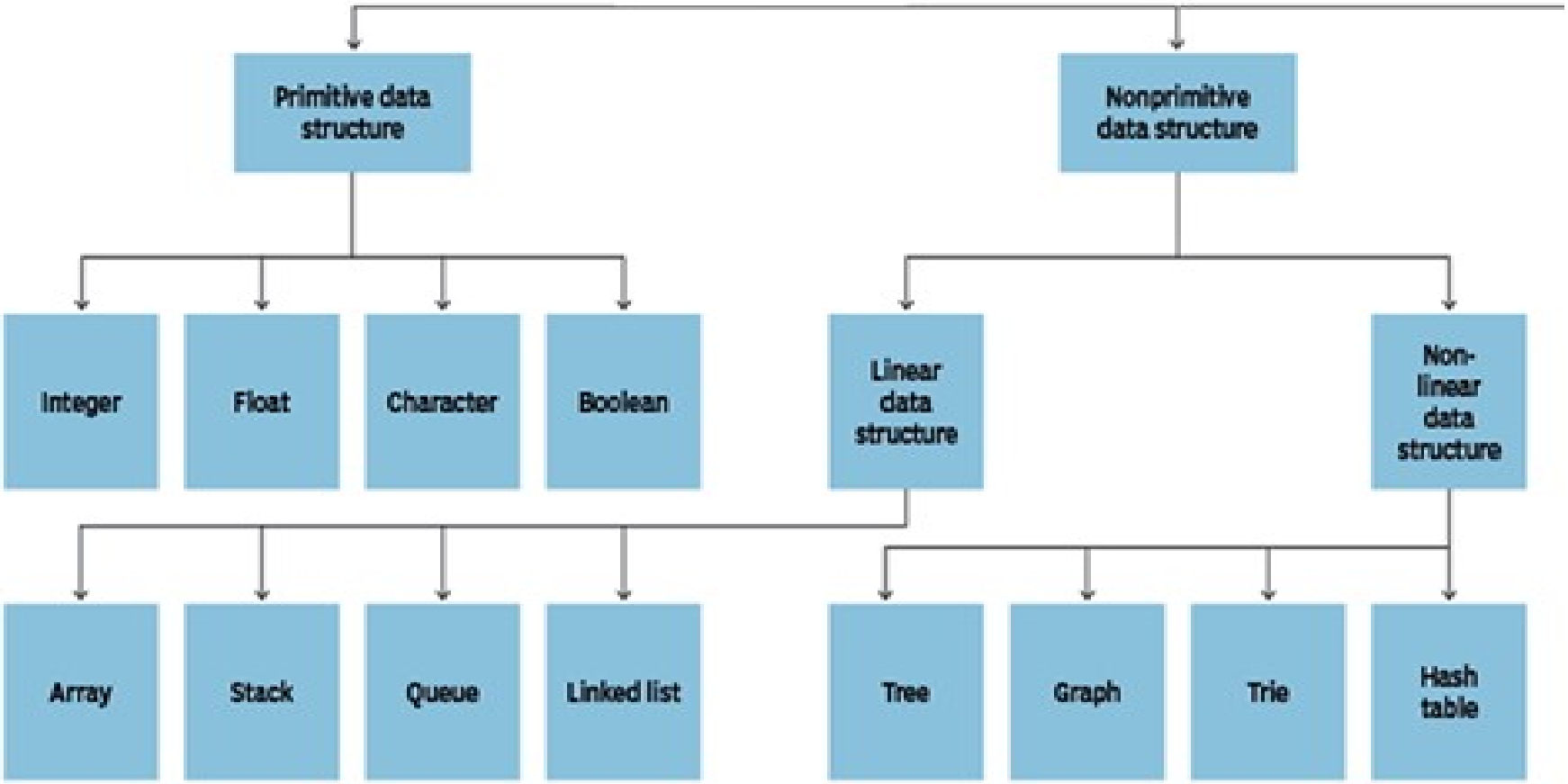
Data Structures - Hierarchy

(classification / Types)



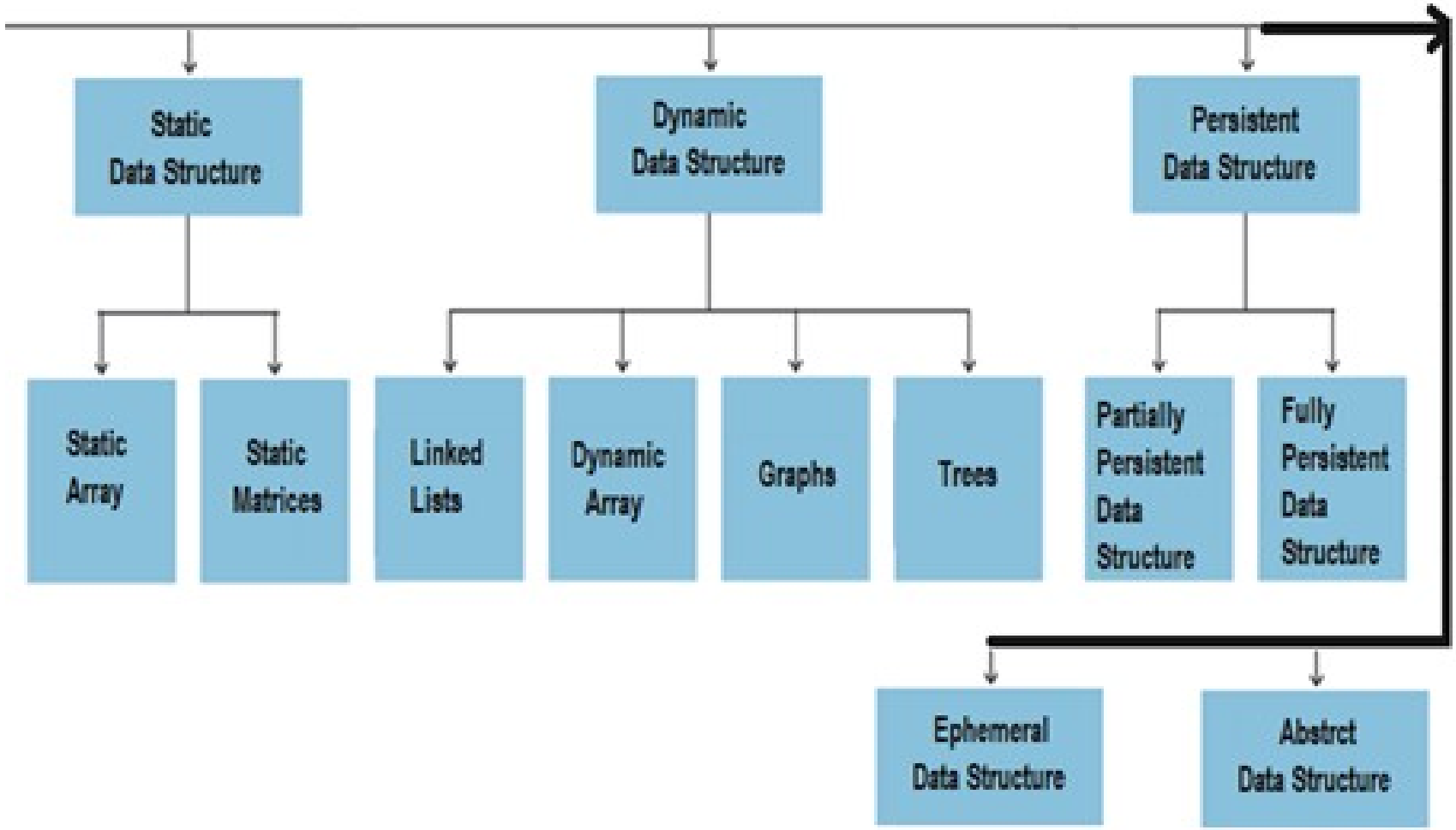
Data Structures - Hierarchy

(classification / Types)



Data Structures - Hierarchy

(classification / Types)



Data Structures Hierarchy

Primitive Data Structures: Data Types

- DS are the building blocks of algorithms and computer programs.
- Primitive (base) data types are the building blocks of data structures.
  - **Boolean,** which stores logical values that are either true or false.
  - **integer,** which stores a range on mathematical integers -- or counting numbers. Different sized integers hold a different range of values -- e.g., a signed 8-bit integer holds values from -128 to 127, and an unsigned long 32-bit integer holds values from 0 to 4,294,967,295.
  - **Fixed-point numbers,** which are used in some programming languages and hold real values but are managed as digits to the left and the right of the decimal point.

Data Structures Hierarchy

Primitive Data Structures: Data Types

- **Character,** which uses symbols from a defined mapping of integer values to symbols.
- **Pointers,** which are reference values that point to other values.
- **String,** which is an array of characters followed by a stop code -- usually a "0" value -- or is managed using a length field that is an integer value.

Data Structure Types

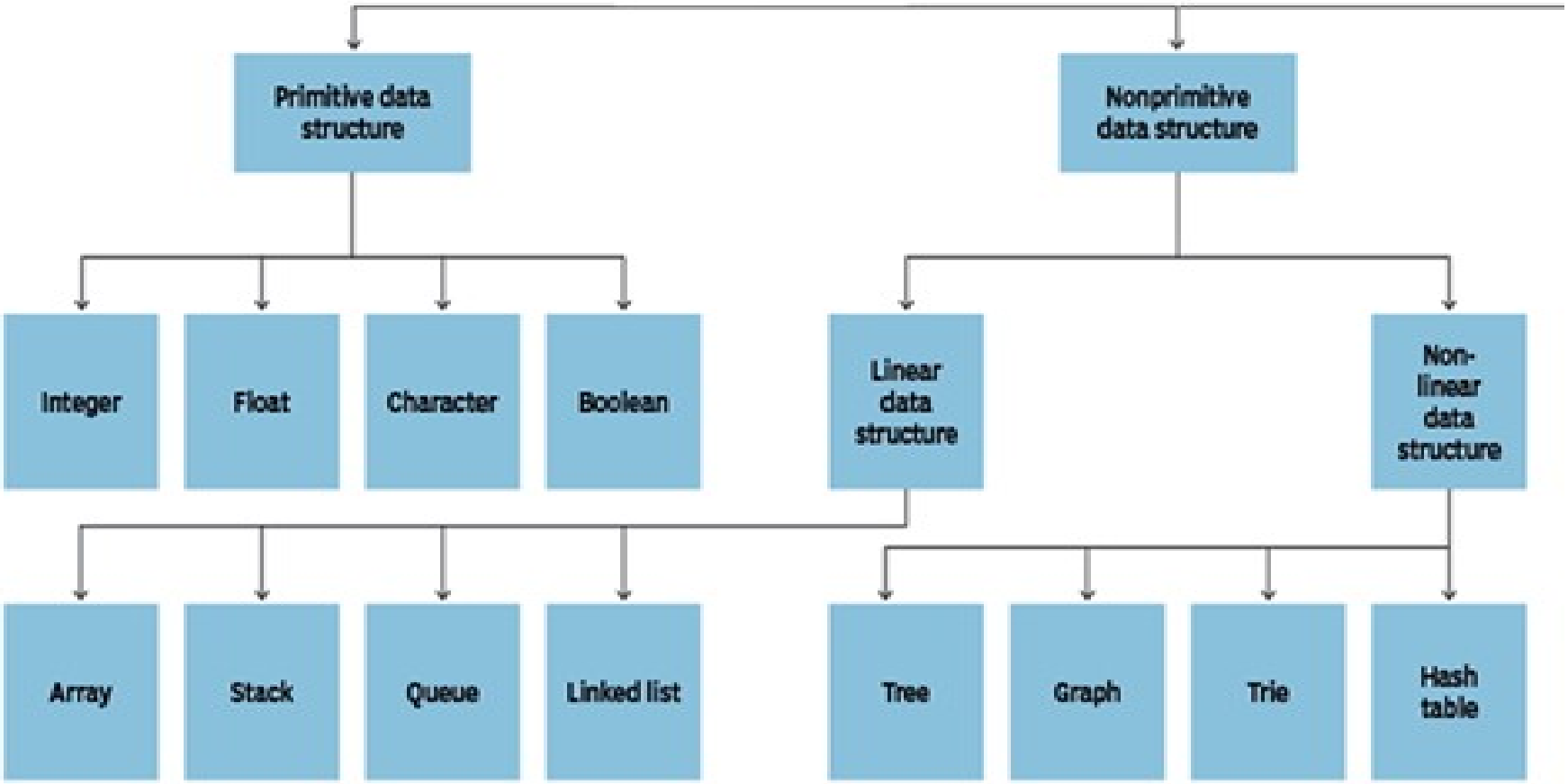
- Linear and Nonlinear (Non-Primitive)
- Static and Dynamic
- Persistent and Ephemeral data structures

Data Structures - Characteristics

- Data structures are often classified by their characteristics.
- The following three characteristics:
  - **Linear or non-linear.** This characteristic describes whether the data items are arranged in sequential order, such as with an array, or in an unordered sequence, such as with a graph.
  - **Homogeneous or heterogeneous.** This characteristic describes whether all data items in a given repository are of the same type. One example is a collection of elements in an array, or of various types, such as an abstract data type defined as a structure in C, a class specification in C++/ Java.
  - **Static or dynamic.** This characteristic describes how the data structures are compiled. Static data structures have fixed sizes, structures and memory locations at compile time. Dynamic data structures have sizes, structures and memory locations that can shrink or expand, depending on the use.

Data Structures - Hierarchy

(classification / Types)



Data Structures Hierarchy

Non-Primitive Data Structures: Linear

- In Linear data structure, values are arrange in linear fashion.
- The data structure type used in a particular situation is determined by **the type of operations** that will be **required** or the **kinds of algorithms** that will be **applied.**
  - Array: Fixed-size static int name[10]
  - Linked-list: Variable-size dynamic pointer int *p; p=&q
  - Stack: Add to top and remove from top
  - Queue: Add to back and remove from front
    - Priority queue: Add anywhere, remove the highest priority

Data Structures Hierarchy

Non-Primitive Data Structures: Linear

Advantages

- Transverse in single run
- Easy to impact

Disadvantage

- Not utilize memory efficiently
- Time complexity increases as size increases

Data Structures Hierarchy

Non-Primitive Data Structures: Linear

- Homogenous: In this type of data structures, values of the same types of data are stored.
  - Array int, float, char

- Non-Homogenous: In this type of data structures, data values of different types are grouped and stored.
  - Structures

```
struct emp
 { int  eid,
        float sal,
        char ename[25]
   }
```

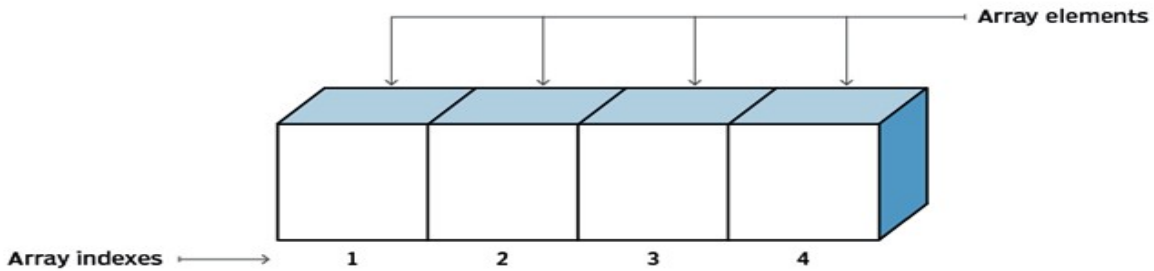
  - Classes

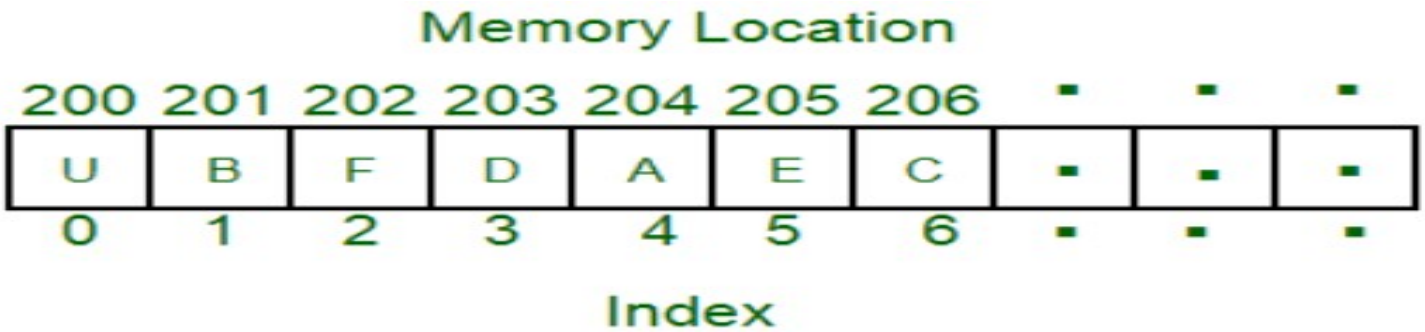
Data Type	Data Structure
The data type is the form of a variable to which a value can be assigned. It defines that the particular variable will assign the values of the given data type only.	Data structure is a collection of different kinds of data. That entire data can be represented using an object and can be used throughout the program.
It can hold value but not data. Therefore, it is dataless.	It can hold multiple types of data within a single object.
The implementation of a data type is known as abstract implementation.	Data structure implementation is known as concrete implementation.
There is no time complexity in the case of data types.	In data structure objects, time complexity plays an important role.
In the case of data types, the value of data is not stored because it only represents the type of data that can be stored.	While in the case of data structures, the data and its value acquire the space in the computer's main memory. Also, a data structure can hold different kinds and types of data within one single object.
Data type examples are int, float, double, etc.	Data structure examples are stack, queue, tree, etc.

Data Structures Hierarchy

Non-Primitive Data Structures: Linear

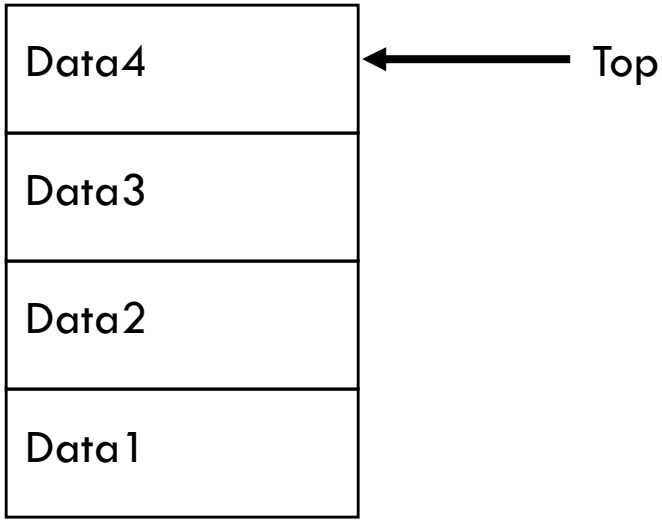
- **Array:**
  - An array stores a collection of items at adjoining memory locations.
  - Items that are the same type are stored together so the position of each element can be calculated or retrieved easily by an index.
  - Arrays can be fixed or flexible in length.





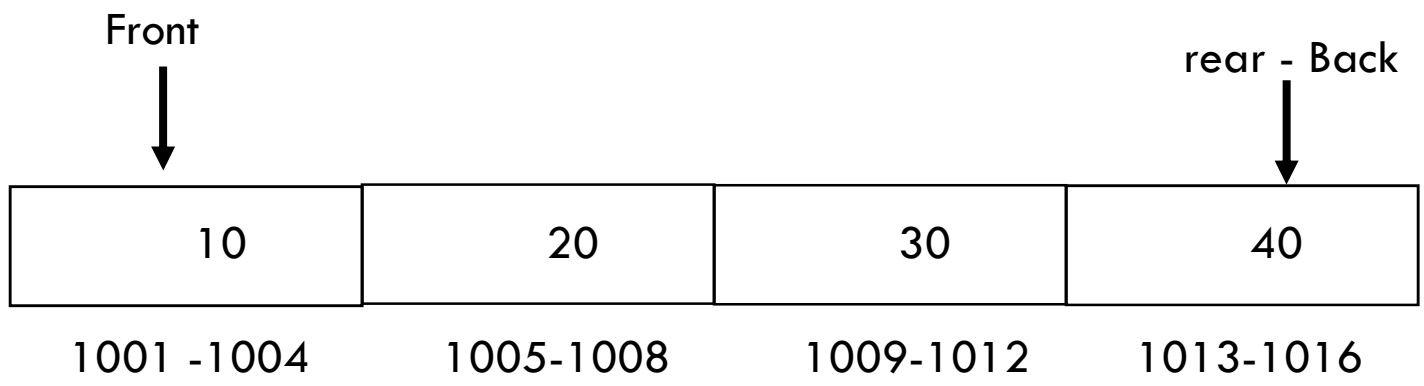
Stacks

- It stores a collection of items in the linear order that operations are applied. This order could be last in, first out (LIFO) or first in, first out (FIFO).
- Collection with access only to the last element inserted
- Last in first out
- LIFO / FILO
- insert/push
- remove/pop
- top
- make empty
- Is full



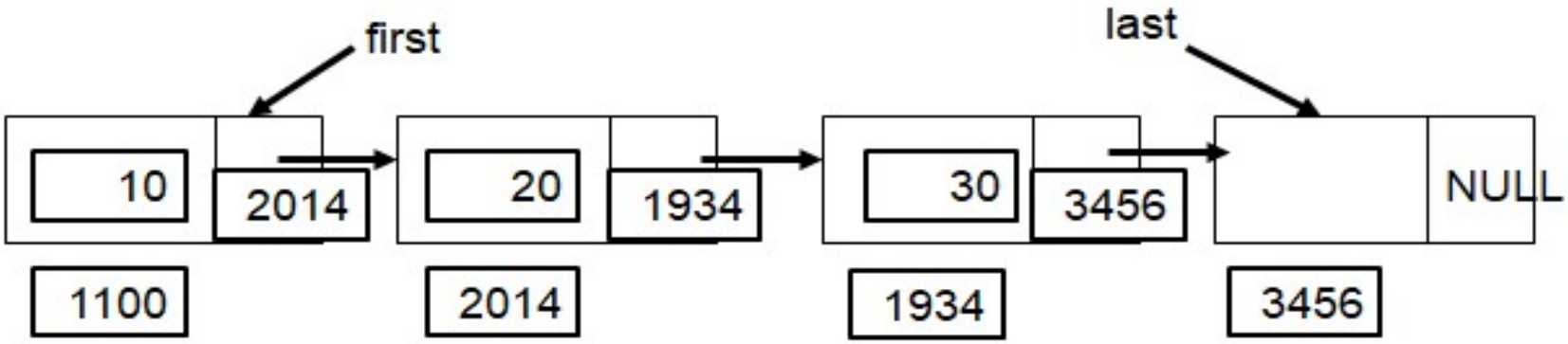
Queues

- It stores a collection of items like a stack; however, the operation order can only be first in, first out. (FIFO)
- Collection with access only to the item that has been present the longest
- First in first out or Last in last out.
- Enqueue -rear, dequeue- front
- priority queues and dequeue



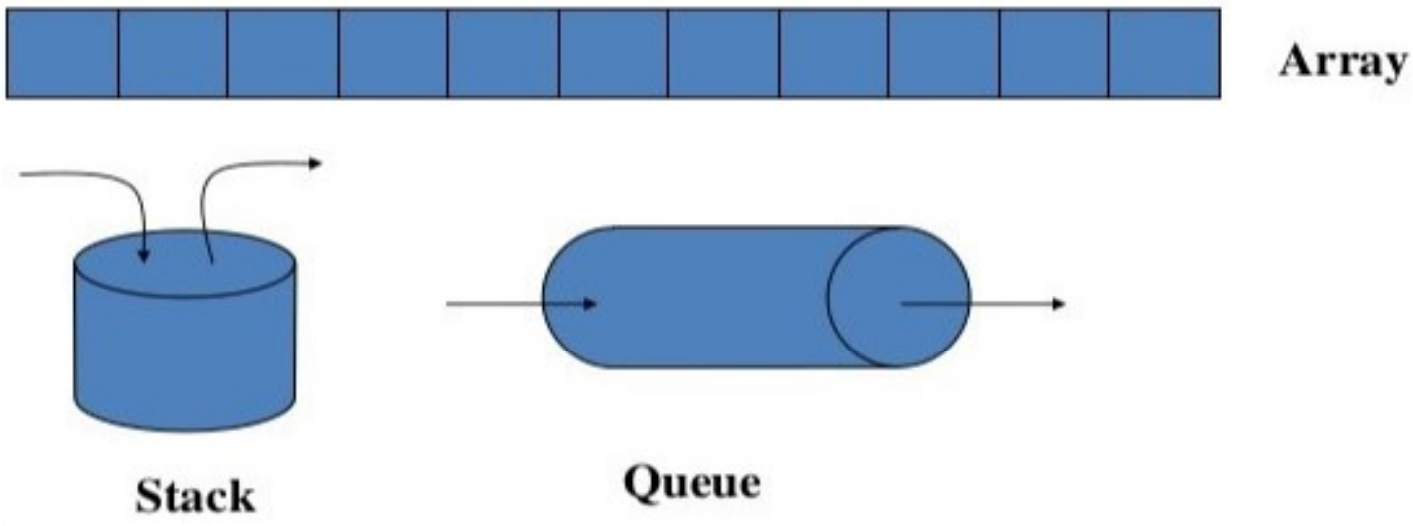
List

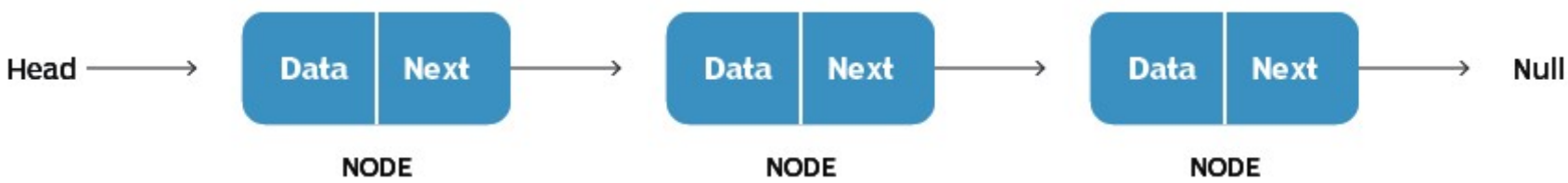
- It stores a collection of items in a linear order. Each element, or node, in a linked list contains a data item, as well as a reference, or link, to the next item in the list.
- A ***Flexible*** structure, because can grow and shrink on demand.
- At ***any*** position Elements can be:
  - Inserted
  - Accessed
  - Deleted



Data Structures Hierarchy

Non-Primitive Data Structures: Linear

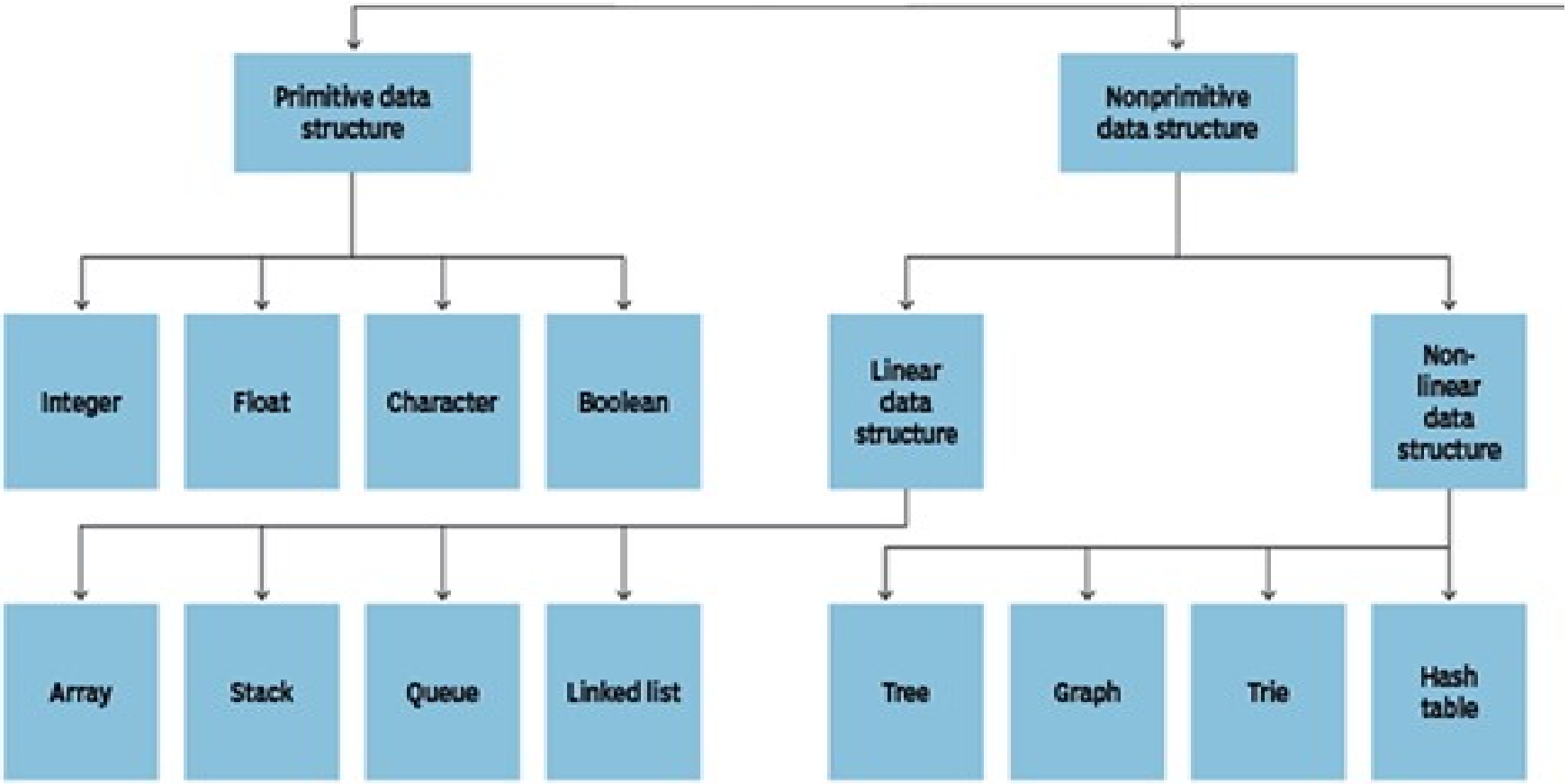




Linked list

Data Structures - Hierarchy

(classification / Types)



Data Structures Hierarchy

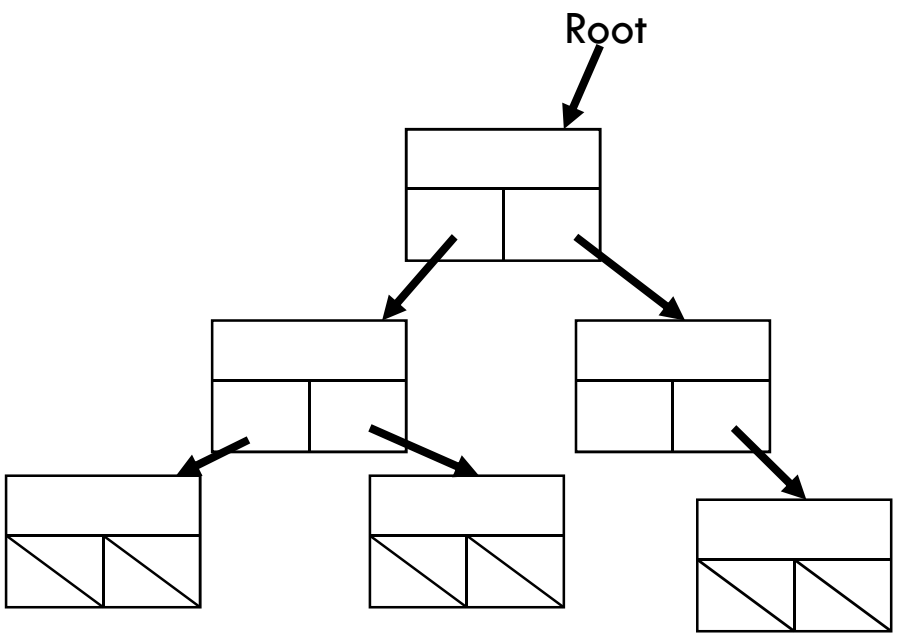
Non-Primitive Data Structures: Non-Linear

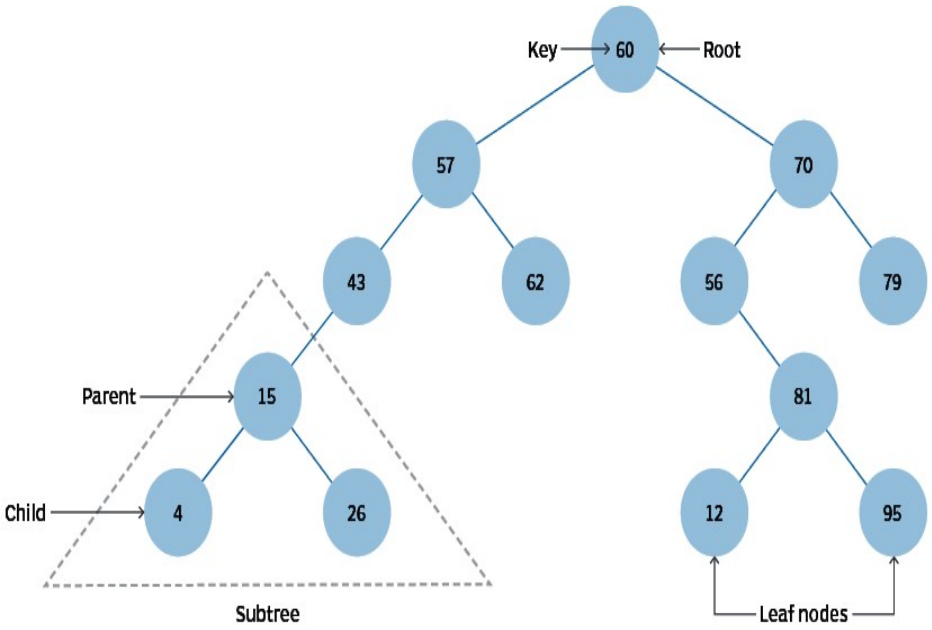
- In Non-Linear data structure, the data values are not arranged in order.
  - **Tree**
  - **Graph**
  - **Hash tables:**
  - **Trie**
  - **Heap**

- **Advantages**
  - Utilize more efficiently
  - Time Complexity remains same as size increases
- **Disadvantages**
  - Transverse in single run not possible
  - Not easy to implement

Tree

- A **Tree** is a collection of elements called **nodes** placed in a hierarchical structure.
- One of the node is distinguished as a **root**, There is one root node that is the ancestor of all the nodes in tree.
- Each node is associated with a key value, with parent nodes linked to child nodes or subnodes.

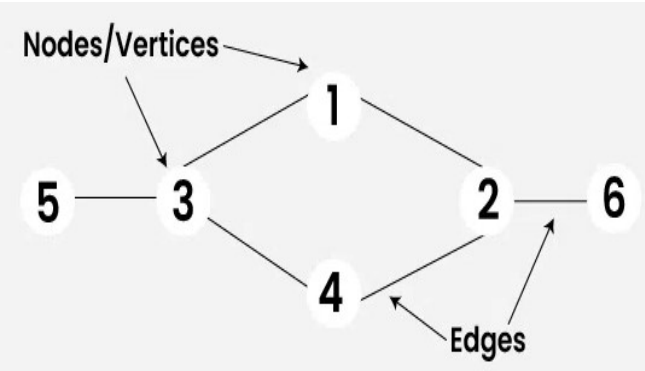


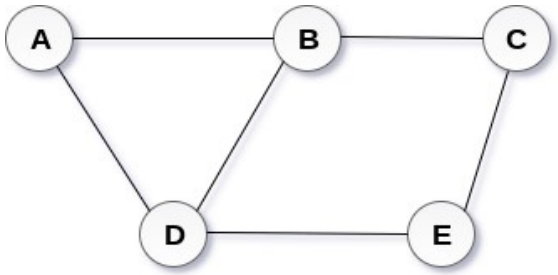


Binary Search Tree

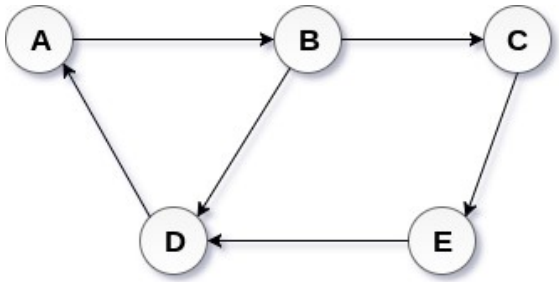
Graph

- It stores a collection of items in a nonlinear fashion.
- Graphs are made up of a finite set of nodes, also known as vertices, and lines that connect them, also known as edges.
- These are useful for representing real-world systems such as computer networks.
- Solving various problems like finding the shortest path or detecting cycles.





Undirected Graph

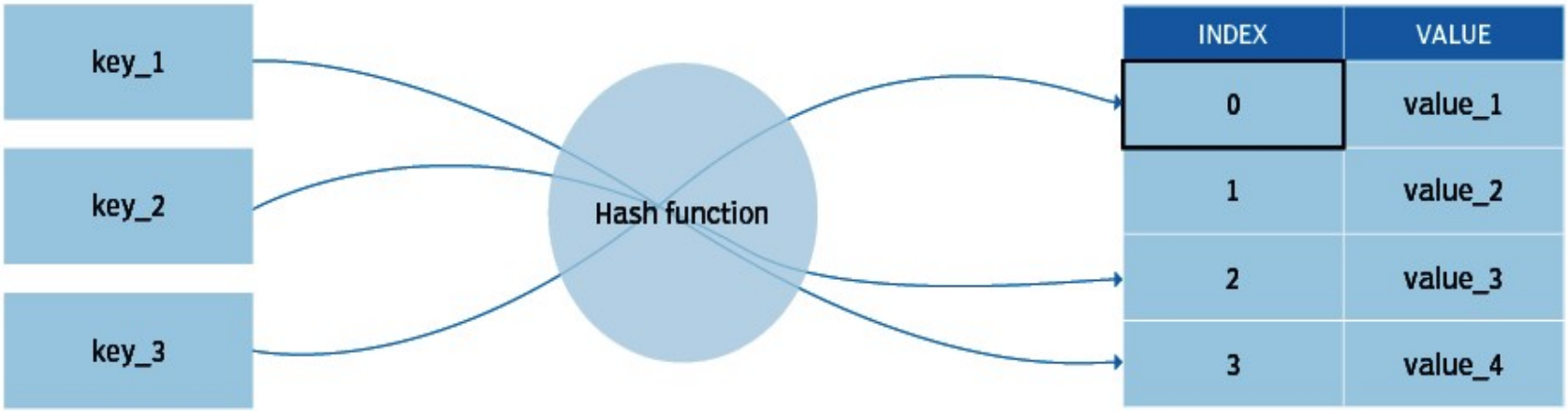


Directed Graph

Hash table

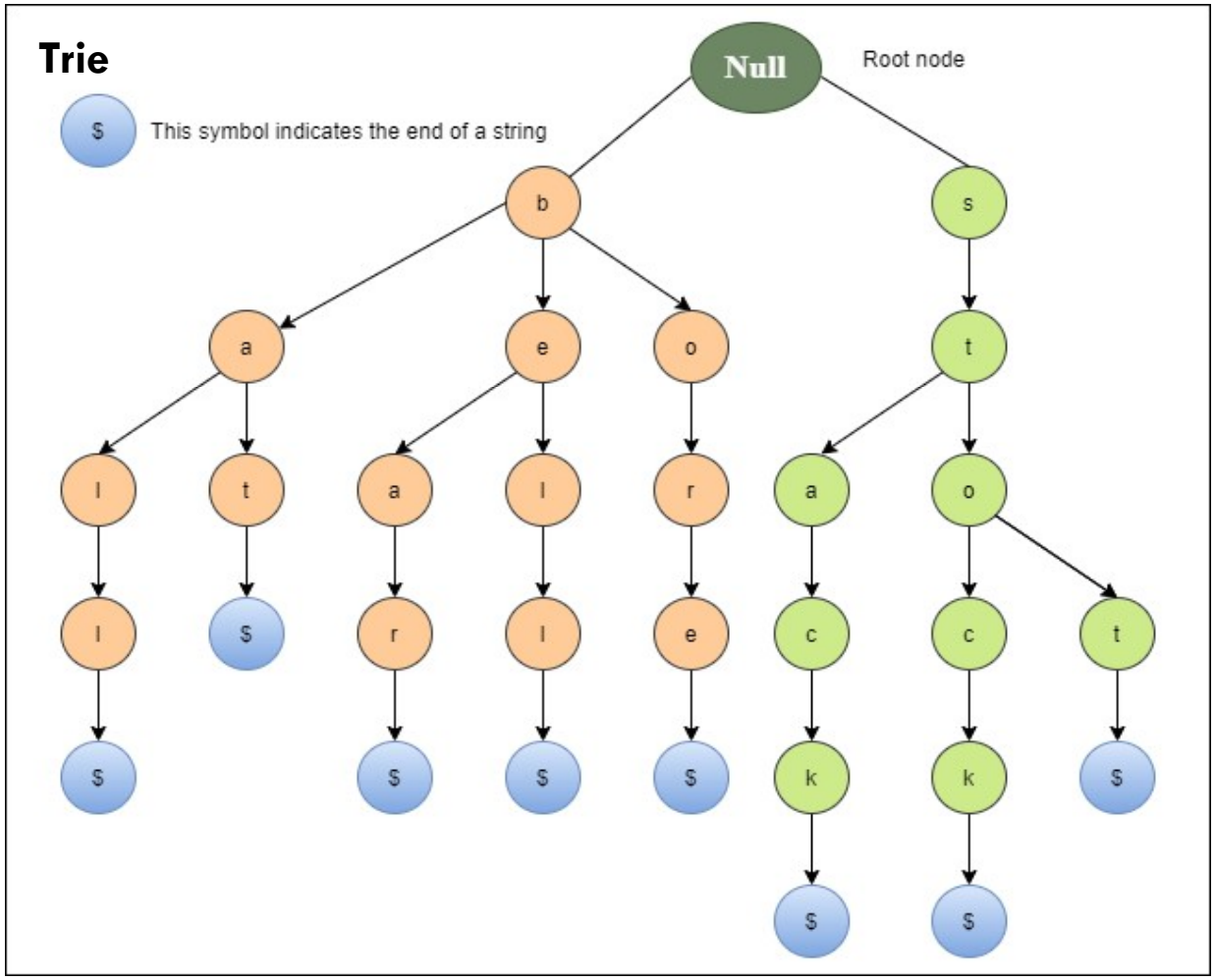
Its also known as a hash map –

- Stores a collection of items in an associative array that plots keys to values.
- A hash table uses a hash function to convert an index into an array of buckets that contain the desired data item.



Trie (Content Beyond Syllabus)

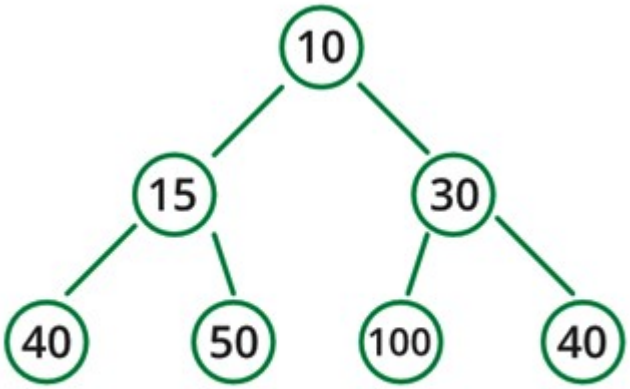
- It is a sorted tree-based data-structure that stores the set of strings used for **retrieval**
- It can search a word in the dictionary with the help of the word's prefix.
- For efficient **retrieval & storage** of keys in a large dataset.
- It supports operations such as **insertion, search,** and **deletion** of keys



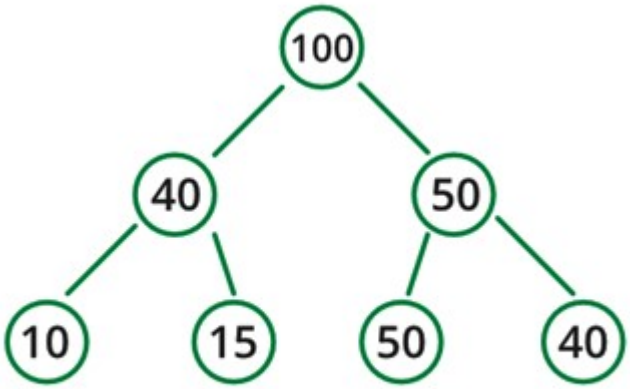
ball bat beat bell bore stack stock stot

Heap

- It is a tree-based structure in which each parent node's associated key value is greater than or equal to the key values of any of its children's key values.
- This property makes sure that root node contains **maximum**/**minimum** value (depending on the type of heap), and the values decrease or increase as move down the tree.



Min Heap



Max Heap

Linear Vs Non-Linear Data Structure

LINEAR DATA STRUCTURES	NON-LINEAR DATA STRUCTURES
Linear Data structures are used to represent **sequential** data.	Non-linear data structures are used to represent **hierarchical** data.
Linear data structures are **easy** to implement	These data structures are **difficult** to implement.
Implementation: Linear data structures are implemented using array and linked lists	**Implementation:** Non-linear data structures are mostly implemented using linked lists.
e.g: The basic linear data structures are list, **stack and queue.**	**e.g:** The basic non-linear data structures are **trees and graphs.**
For the implementation of linear data structures, we don't need **non-linear** data structures.	For the implementation of non-linear data structures, we need **linear** data structures.
USE: These are mostly used in application software development.	**USE:** These are used for the development of game theory, artificial intelligence, image processing

Static Data Structure

- The **size is fixed** which means that the size of that data structure cannot be change that is defined in the program.
- Example: School timetable - classes and lunch breaks mentioned in that. So all the classes had fix timing or duration the same way during coding static data structure used which has a fix size.
- **Types of Static Data Structure**
  - Static Array
  - Static Matrices
- **Advantages**
  - Simplicity
  - More Efficiently runs the program
  - Fixed Size Limitation

* **Disadvantages**

- Fixed Size Limitation
- Inflexible

Dynamic Data Structure

- The data structure who's size is not fixed and can be modified during the operation performed on it.
- Example: Let's make the concept of this data structure very easy. You all have taken pics or photo's on your phones. When you zoom in the image, it changes it's view and when you zoom out then it comes back to the original form. So the changing of size of the image without changing the property of the image and affecting it is dimension is called as dynamic change or dynamic image. The same way when a data structure changes it's size as per the program is know as Dynamic Data Structure.
- **Types**
  - Linked Lists
  - Dynamic Array
  - Graphs
  - Tree
- **Advantages**
  - Flexible
  - No Memory Wastage
- **Disadvantages**
  - Complexity Increases
  - Potentially Slower Access

Persistent Data Structure

- Always **preserves the previous version** of itself when its modified(immutable)
- Example: - Linked List, Tree

  While Purchasing a new smartphone you keep the old or previous smartphone with you for the backup in case if something goes with the new smartphone. In the same way, the data structure which keeps it's previous version after getting modified is called as Persistent Data Structure.
- **Types of Persistent Data Structure**
  - Partially Persistent Data Structure
  - Fully Persistent Data Structure
- **Advantage**
  - Keeps track of previous version
  - Holds multiple version of data which allows time traveling
- **Disadvantage**
  - Increase Memory Overhead
  - More Complex which increases the complexity

Ephemeral Data Structure

- The data structure which has only one version available at a time and no other previous version like Persistent Data Structure has.
- Example : RAM, cache memory

  When the recycle bin is cleaned and remove the file permanently which once done can't be undo OR

  When you delete a file permanently from your system which can't be undo once done.
- **Advantages**
  - Simpler to implement
  - Reduce Memory usage
- **Disadvantage**
  - Do not keep track of previous version
  - Do not support undo/redo (Limited to some application)

Abstract Data Structure (ADT)

Definitions:

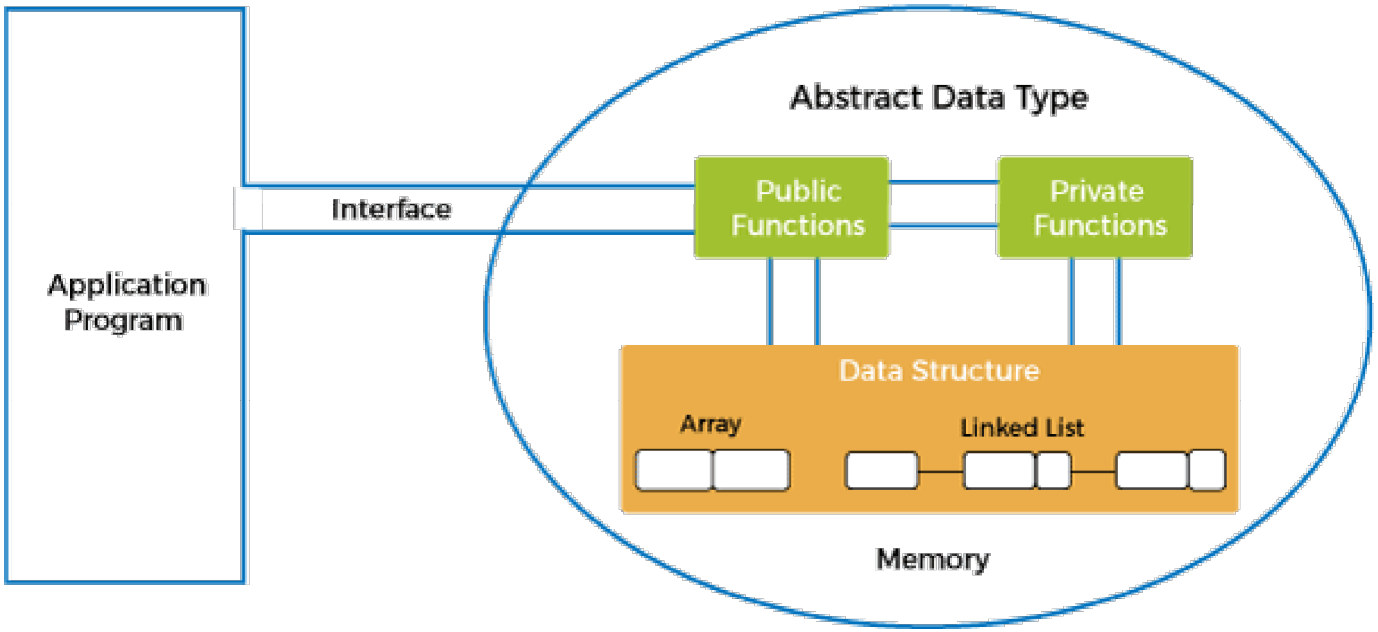
- **ADTs are a way of encapsulating data and operations on that data into a single unit.**
- It is a collection of data and associated operations for manipulating that data
- ADT is an **abstraction** of a data structure that **provides only** the **interface** to which the data structure must adhere.
- Defines of data and operations but do **not give any** implementation details or hides the implementation details.
- It stores data and allow various operations on the data to access & change it.
- There are two main **Components** in ADT:
  - Data
  - Operations
- Examples of ADT are : **Stack** ADT , **Queue** ADT , **List** ADT etc.

Abstract Data Type and Data Structure

- **Data Structures**
  - Physical implementation of an ADT
  - data structures used in implementations are provided in a language (primitive or built-in) or are built from the language constructs (user-defined)
  - Each operation associated with the ADT is implemented by one or more subroutines in the implementation

Abstract Data Type - ADT

- ADTs support *abstraction*, *encapsulation*, and *information hiding*.
- *Abstraction* is the structuring of a problem into well-defined entities by defining their data and operations.
- The principle of hiding the used data structure and to only provide a well-defined interface is known as *encapsulation*.



Abstract Data Type - Features

- **Abstraction:** The user does not need to know the implementation of the data structure only essentials are provided.
- **Better Conceptualization:** ADT gives us a better conceptualization of the real world.
- **Robust:** The program is robust and has the ability to catch errors.
- **Encapsulation**: ADTs hide the internal details of the data and provide a public interface for users to interact with the data. This allows for easier maintenance and modification of the data structure.
- **Data Abstraction**: ADTs provide a level of abstraction from the implementation details of the data. Users only need to know the operations that can be performed on the data, not how those operations are implemented.

Abstract Data Type - Features

- **Data Structure Independence:** ADTs can be implemented using different data structures, such as arrays or linked lists, without affecting the functionality of the ADT.
- **Information Hiding:** ADTs can protect the integrity of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
- **Modularity:** ADTs can be combined with other ADTs to form larger, more complex data structures. This allows for greater flexibility and modularity in programming.
- **Reusability:** multiple client programs can use the data structure.

Abstract Data Type - Advantages

- **Encapsulation**: ADTs provide a way to encapsulate data and operations into a single unit, making it easier to manage and modify the data structure.
- **Abstraction**: ADTs allow users to work with data structures without having to know the implementation details, which can simplify programming and reduce errors.
- **Data Structure Independence**: ADTs can be implemented using different data structures, which can make it easier to adapt to changing needs and requirements.
- **Information Hiding**: ADTs can protect the integrity of data by controlling access and preventing unauthorized modifications.
- **Modularity**: ADTs can be combined with other ADTs to form more complex data structures, which can increase flexibility & modularity in programming.

Abstract Data Type - Disadvantages

- **Overhead**: Implementing ADTs can add overhead in terms of memory and processing, which can affect performance.
- **Complexity**: ADTs can be complex to implement, especially for large and complex data structures.
- **Learning** Curve: Using ADTs requires knowledge of their implementation and usage, which can take time and effort to learn.
- **Limited Flexibility:** Some ADTs may be limited in their functionality or may not be suitable for all types of data structures.
- **Cost**: Implementing ADTs may require additional resources and investment, which can increase the cost of development.

Data Structures - Core Operations

- The major or the common operations that can be performed on the data structures are:
  - **Insertion:** to insert the new element in a data structure.
  - **Updation:** to update the element, i.e., to replace the element with another element.
  - **Deletion:** to remove the element from the data structure.
  - **Searching:** to search for any element in a data structure.
  - **Sorting:** to sort the elements of a data structure either in an ascending or descending order.
  - **Traversing:** visiting each element stored in the data structure in order to perform some specific operation like searching or sorting.
  - **Merging:** Combining from two diff. sorted list into a single list.
    Let List A and List B of size M and N resp., of similar type of elements, clubbed or joined to produce the third list, List C of size (M+N).

Data Structures - Core Operations

- Every Collection ADT should provide a way to:
  - add an item
  - remove an item
  - find, retrieve, or access an item
- Many, many more possibilities
  - is the collection empty
  - make the collection empty
  - give me a sub set of the collection 12345678910
  - 1 3 5 7 9 2 4 6 8 10
- No single data structure works well for all purposes, and so it is important to know the strengths and limitations of several of them
- Int 10 52 6888 float 53.44 char class[] ="DSE" array

Topics

Introduction:

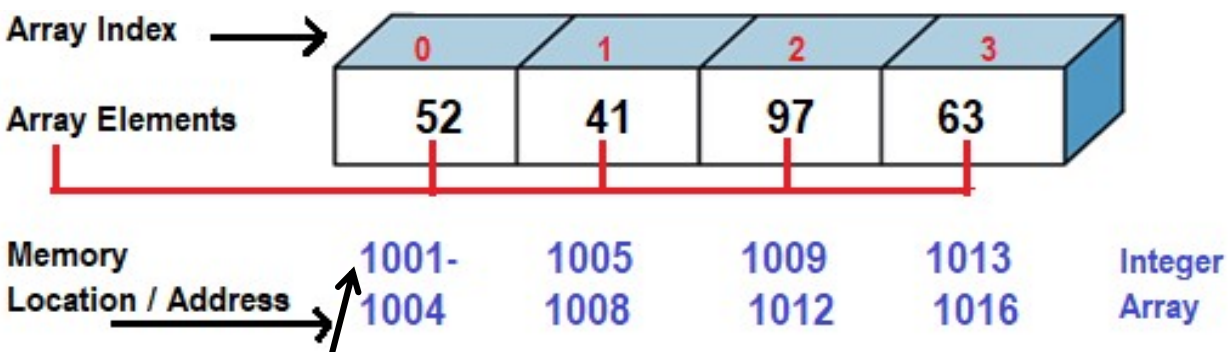
- Problem Solving –
  - From Problem to Program - Problem, Solution, Algorithm, Data Structure & Program
- Types of Data Structures
  - Linear & Nonlinear , Static & Dynamic, Persistent & Ephemeral data structures
- Concept of ADT
- Operations on Data Structures.

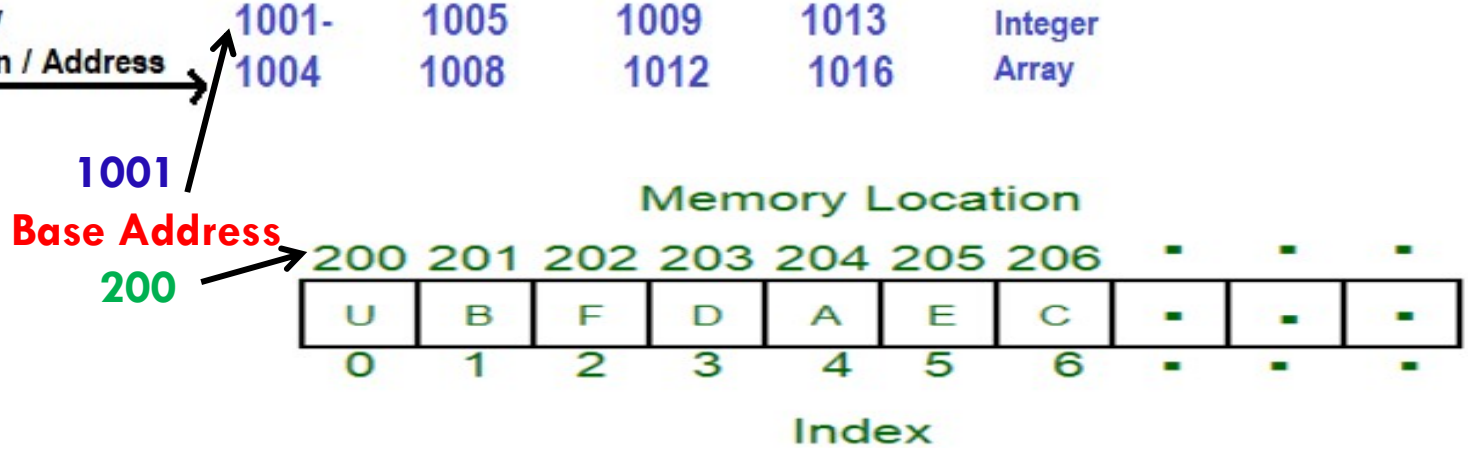
Arrays:

- Arrays as ADT, representation of Linear array in Memory
- Traversing Linear Arrays
- Inserting and deleting.

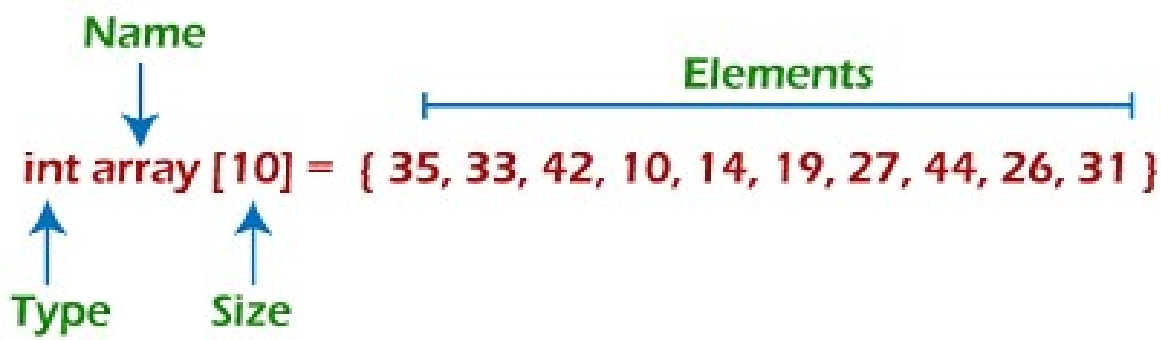
Array as ADT

- **Stores multiple values in a single variable.**
- An array stores a collection of items at adjoining memory locations.
- Items that are the same type are stored together so the position of each element can be calculated or retrieved easily by an index.





Arrays

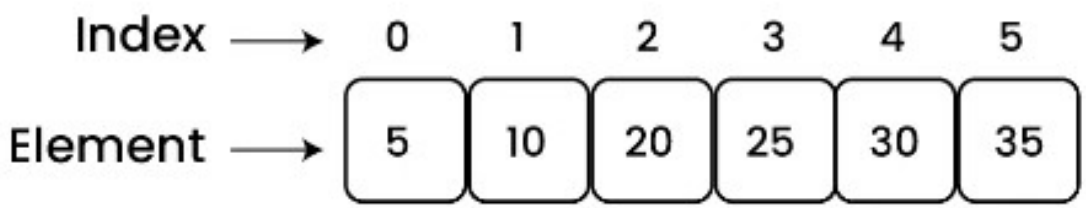


Types of Array

- **One-dimensional arrays:** These arrays store a single row of elements.
- **Multidimensional arrays:** These arrays store multiple rows of elements.

Arrays

- **One-dimensional arrays:** These arrays store a single row of elements.



- **Multidimensional arrays: A**rrays with more than one dimension.
- 2-D arrays, 3-D arrays, 4-D arrays and so on.

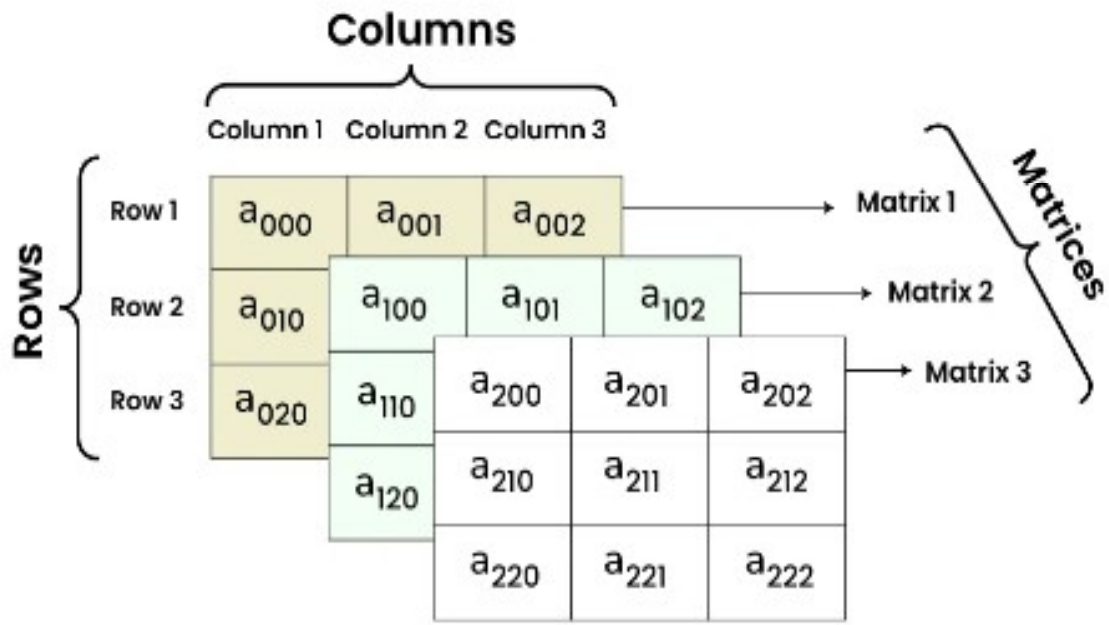
Arrays

Two-Dimensional Array (2-D Array or Matrix)

Columns →

Rows ↓	0	1	2
0	a_{00}	a_{01}	a_{02}
1	a_{10}	a_{11}	a_{12}
2	a_{20}	a_{21}	a_{22}

Three-Dimensional Array (3-D Array)



Operations on Arrays

Following are the basic operations supported by an array.

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Sorting** – rearrange of elements in a specific order (Ascending/descending)
- **Display** – Displays the contents of the array.

(refer 1. 2 Array operations.docx file for practice)

Arrays - Advantages

- Arrays allow **random access** to elements. This makes accessing elements by position faster.
- Arrays have **better cache locality** which makes a pretty big difference in performance.
- Arrays **represent multiple data items of the same type** using a single name.
- Arrays are used to implement the other data structures like linked lists, stacks, queues, trees, graphs, etc.

Arrays - Disadvantages

- As arrays have a fixed size, once the memory is allocated to them, it cannot be increased or decreased, making it impossible to store extra data if required. An array of fixed size is referred to as a static array.
- Allocating less memory than required to an array leads to loss of data.
- An array is homogeneous in nature so, a single array cannot store values of different data types.
- Arrays store data in contiguous memory locations, which makes deletion and insertion very difficult to implement. This problem is overcome by implementing linked lists, which allow elements to be accessed sequentially.

Arrays - Applications

- They are used in the implementation of other data structures such as array lists, heaps, hash tables, vectors, and matrices.
- Database records are usually implemented as arrays.
- It is used in lookup tables by computer.

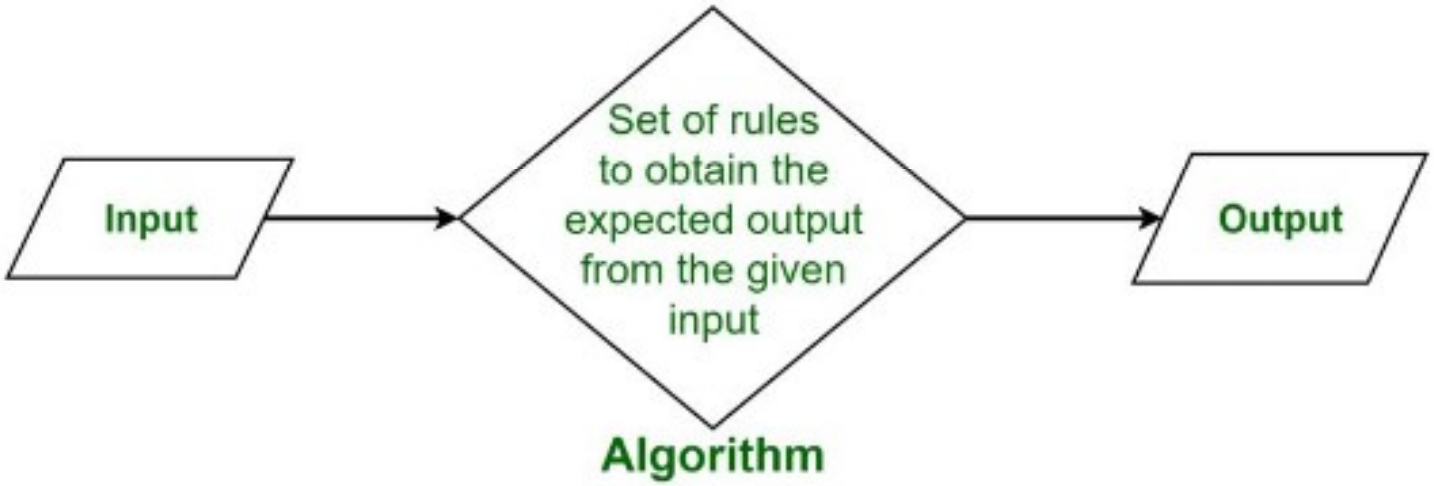
Topics (...continued)

Algorithms:

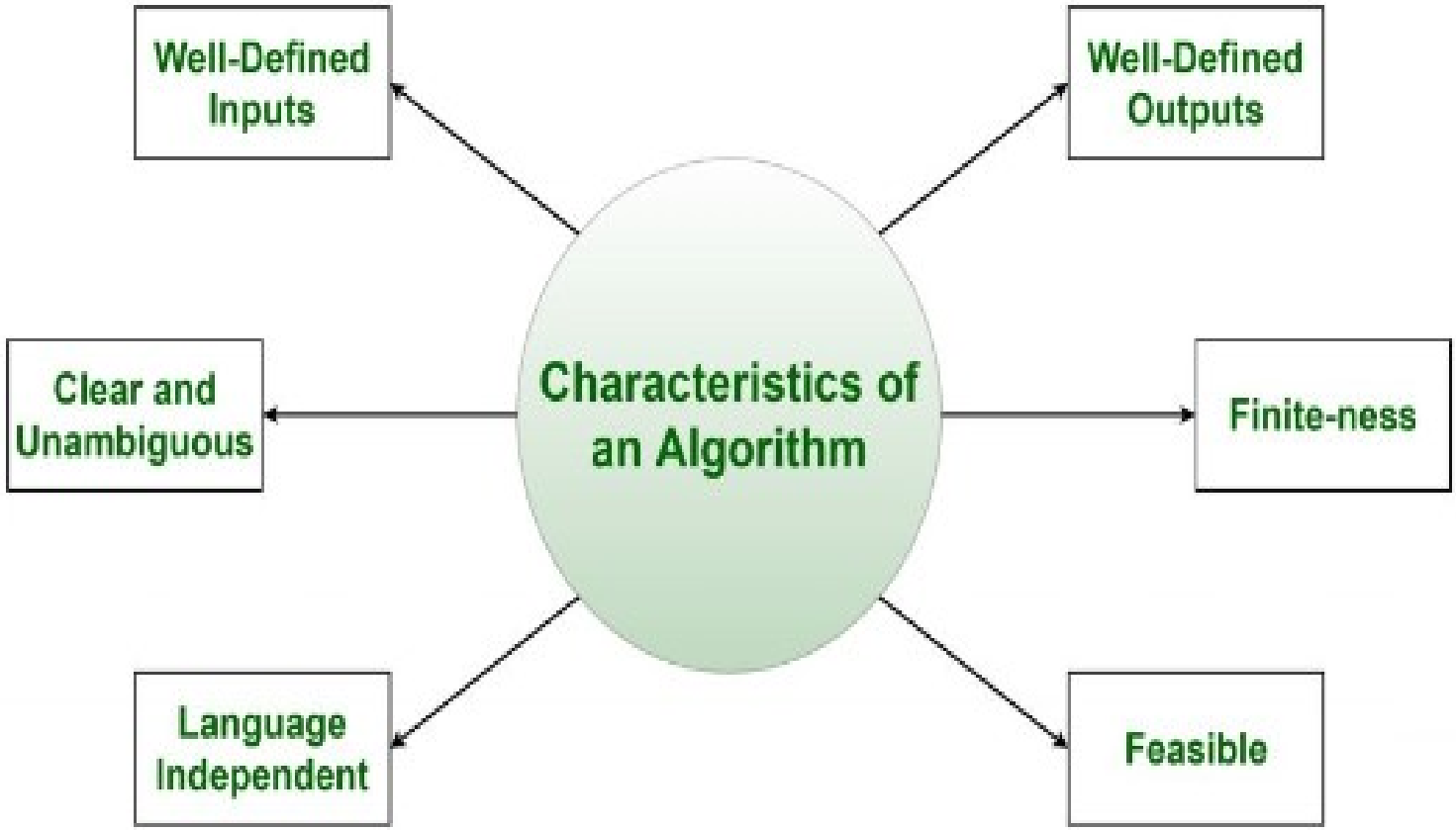
- Introduction to algorithm,
- Characteristics of algorithm,
- Pseudo-code and flowchart,
- Space complexity, time complexity,
- Order of growth,
- Asymptotic notation- big-o, theta and omega,
- Finding complexity using step count method.

Algorithms

- A set of finite rules or instructions to be followed in calculations or other problem-solving operations



Algorithms - Characteristics



Algorithms – Characteristics (Qualities)

- **Clear and Unambiguous**: The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only **one meaning**.
- **Well-Defined Inputs**: the **type, format,** and **range** of **acceptable** input data for the algorithm must be clearly specified.
- **Well-Defined Outputs:** For any given **valid input**, the algorithm should produce a **specific and predictable output,** leaving no room for ambiguity or multiple possibilities. It should produce at least 1 output.
- **Finiteness:** an algorithm **must terminate** after a finite number of steps, meaning it should **not run infinitely**.
- **Feasible:** The algorithm must be simple, generic, and **practical**, such that it can be executed with the **available resources** and **acceptable time**. It must not contain some future technology or anything.
- **Language Independent:** the core logic and steps of an algorithm can be expressed and implemented in any programming language and yet the output will be the same, as expected.

Algorithms – Properties

more specific attributes or features

- **Correctness:** The algorithm produces the correct output for all valid inputs.
- **Efficiency:** The algorithm uses resources (time, space) efficiently.
- **Feasibility:** The algorithm can be implemented using available resources.
- **Completeness:** For search algorithms, it means finding a solution if one exists.
- **Optimality:** For search algorithms, it means finding the best solution (if applicable).

Algorithms - Types

1. **Brute Force Algorithm:**

   It is the simplest approach to a problem. A brute force algorithm is the first approach that comes to finding when we see a problem.

2. **Recursive Algorithm:**

   A recursive algorithm is based on recursion. In this case, a problem is broken into several sub-parts and called the same function again and again.

3. **Backtracking Algorithm:**

   It builds solution by searching among all possible solutions. Using this algorithm, we keep on building the solution following criteria. Whenever a solution fails we trace back to the failure point build on the next solution and continue this process till we find the solution or all possible solutions are looked after.

4. **Searching Algorithm:**

   Searching algorithms are the ones that are used for searching elements or groups of elements from a particular data structure. They can be of different types based on their approach or the data structure in which the element should be found.

Algorithms - Types

5. Sorting Algorithm:

Sorting is arranging a group of data in a particular manner according to the requirement. The algorithms which help in performing this function are called sorting algorithms. Generally sorting algorithms are used to sort groups of data in an increasing or decreasing manner.

6. Hashing Algorithm:

Hashing algorithms work similarly to the searching algorithm. But they contain an index with a key ID. In hashing, a key is assigned to specific data.

7. Divide and Conquer Algorithm:

This algorithm breaks a problem into sub-problems, solves a single sub-problem, and merges the solutions to get the final solution. It consists of the following three steps:

Divide

Solve

Combine

Algorithms - Types

8. Greedy Algorithm:

In this type of algorithm, the solution is built part by part. The solution for the next part is built based on the immediate benefit of the next part. The one solution that gives the most benefit will be chosen as the solution for the next part.

9. Dynamic Programming Algorithm:

This algorithm uses the concept of using the already found solution to avoid repetitive calculation of the same part of the problem. It divides the problem into smaller overlapping subproblems and solves them.

10. Randomized Algorithm:

In the randomized algorithm, we use a random number so it gives immediate benefit. The random number helps in deciding the expected outcome.

Algorithms – Advantages & Disadvantages

Advantages:

- It is easy to understand.
- An algorithm is a step-wise representation of a solution to a given problem.
- In an Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages:

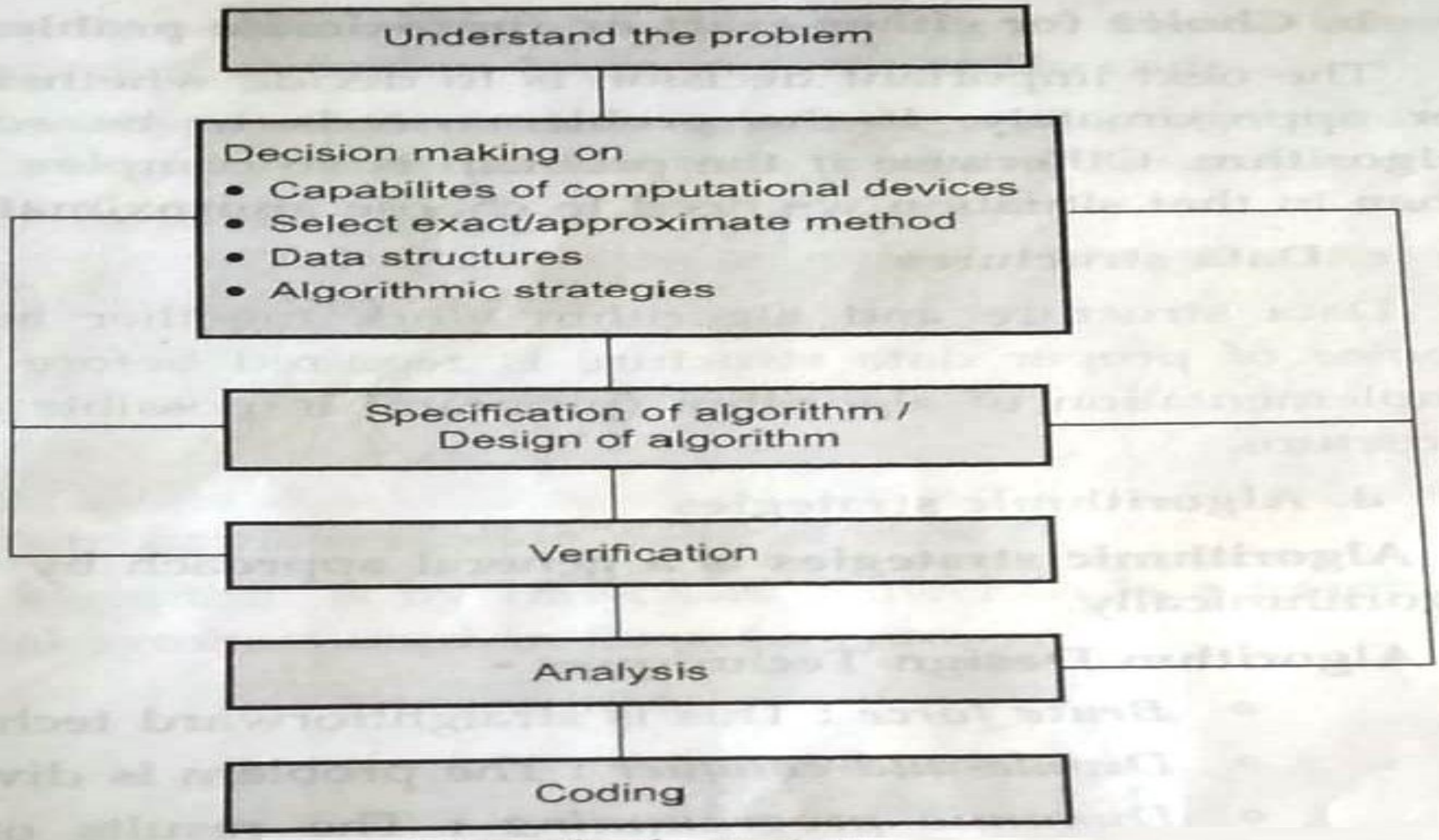
- Writing an algorithm takes a long time so it is time-consuming.
- Understanding complex logic through algorithms can be very difficult.
- Branching and Looping statements are difficult to show in Algorithms**(imp)**.

Algorithms - How to Design

How to Design an Algorithm?

- To write an algorithm, the following things are needed as a pre-requisite:
- The **problem** that is to be solved by this algorithm i.e. clear problem definition.
- The **constraints** of the problem must be considered while solving the problem.
- The **input** to be taken to solve the problem.
- The **output** is to be expected when the problem is solved.
- The **solution** to this problem is within the given constraints.
- Then the algorithm is written with the help of the above parameters such that it solves the problem.

Algorithms - How to Design



Algorithm design steps

Algorithms - Complexity

The two factors of Algorithm Complexity are:

- **Space Factor:** Space is measured by counting the maximum memory space required by the algorithm to run/execute.
- **Time Factor:** Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.

- Complexity of an algorithm can be divided into two types:
  1. **Space Complexity**
  2. **Time Complexity**

Algorithms- Space Complexity

1. **Space Complexity**: The space complexity of an algorithm refers to the amount of memory required by the algorithm to store the variables and get the result. This can be for inputs, temporary operations, or outputs.

- **How to calculate Space Complexity?**
  The space complexity of an algorithm is calculated by determining the following 2 components:
- **Fixed Part:** This refers to the space that is required by the algorithm. For example, input variables, output variables, program size, etc.
- **Variable Part:** This refers to the space that can be different based on the implementation of the algorithm. For example, temporary variables, dynamic memory allocation, recursion stack space, etc.
  Therefore Space complexity **S(P)** of any algorithm P is **S(P) = C + SP(I)**, where C is the fixed part and S(I) is the variable part of the algorithm, which depends on instance characteristic I.

Algorithms- Space Complexity

- *Space taken by the algorithm to solve the problem*. It includes space used by necessary input variables and any extra space (excluding the space taken by inputs) that is used by the algorithm. For example, if we use a hash table (a kind of data structure), we need an array to store values so
- this is an extra space occupied, hence will count towards the space complexity of the algorithm. This extra space is known as Auxiliary Space.
- Space complexity is a function describing the amount of memory(space)an algorithm takes in terms of the amount of input to the algorithm.
- Space complexity is sometimes ignored because the space used is minimal and/ or obvious, but sometimes it becomes an issue as time.

Algorithms- Space Complexity

Example: Consider the below algorithm for Linear Search

- Step 1: START
- Step 2: Get n elements of the array in arr and the number to be searched in x
- Step 3: Start from the leftmost element of arr[] and one by one compare x with each element of arr[]
- Step 4: If x matches with an element, Print True.
- Step 5: If x doesn't match with any of the elements, Print False.
- Step 6: END

There are 2 variables arr[], and x, where the arr[] is the variable part of n elements and x is the fixed part. Hence S(P) = 1+n. So, the space complexity depends on n(number of elements). Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

Algorithms- Space Complexity

$$S(p) = C + Sp$$

```
int a =10, b =25, sum=0;

sum= a+b

print sum
```

c=3 Sp=0

s(p)= 3+0 =3

```
int sum=0,  n , i

for (i=1 ; i<=n . ; i++)
    sum =sum + i

print sum
```

c=3

Sp = 1 for first value of i
=2
=3
.
.
n

S(p) = C + Sp
= 3 +n
S(p) > = (n+3)

Algorithms- Space Complexity

- **Example:** Consider the below algorithm for Linear Search
- Step 1: START
  Step 2: Get n elements of the array in arr and the number to be searched in x
  Step 3: Start from the leftmost element of arr[] and one by one compare x with each element of arr[]
  Step 4: If x matches with an element, Print True.
  Step 5: If x doesn't match with any of the elements, Print False.
  Step 6: END
  Here, There are 2 variables arr[], and x, where the arr[ ] is the variable part of n elements and x is the fixed part. Hence S(P) = 1+n. So, the space complexity depends on n(number of elements). Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

Algorithms- Time Complexity

2. Time Complexity: The time complexity of an algorithm refers to the amount of time required by the algorithm to execute and get the result. This can be for normal operations, conditional if-else statements, loop statements, the number of comparisons between integers, etc.

- **How to Calculate, Time Complexity?**
  It is calculated by determining the following 2 components:
  - **Constant time part:** Any instruction that is executed just once comes in this part. For example, input, output, if-else, switch, arithmetic operations, etc.
  - **Variable Time Part:** Any instruction that is executed more than once, say n times, comes in this part. For example, loops, recursion, etc.
    Therefore Time complexity $T(P)$ of any algorithm P is
  - **T(P) = C + TP(I)**, where C is the constant time part and TP(I) is the variable part of the algorithm, which depends on the instance characteristic I.
- **Time Complexity** can be calculated by using Two types of **methods**.
  - Asymptotic Notation.
  - Step Count Method

Algorithms- Time Complexity

Example: In the algorithm of Linear Search above, the time complexity is calculated as follows:

Step 1: – Constant Time
Step 2: – Variable Time (Taking n inputs)
Step 3: – Variable Time (Till the length of the Array (n) or the index of the found element)
Step 4: – Constant Time
Step 5: – Constant Time
Step 6: – Constant Time

Hence, $T(P) = 1 + n + n(1 + 1) + 1$

$= 2 + 3n$, which can be said as $T(n)$.

$= T(n)$

Algorithms
How to express an Algorithm?

- **Natural Language:-** Here we express the Algorithm in the natural English language. It is too hard to understand the algorithm from it.
- **Flowchart:-** Here we express the Algorithm by making a graphical/pictorial representation of it. It is easier to understand than Natural Language.
- **Pseudo Code:-** Here we express the Algorithm in the form of annotations and informative text written in plain English which is very much similar to the real code but as it has no syntax like any of the programming languages, it can't be compiled or interpreted by the computer. It is the best way to express an algorithm because it can be understood by even a layman with some school-level knowledge.

Algorithms – Time Complexity

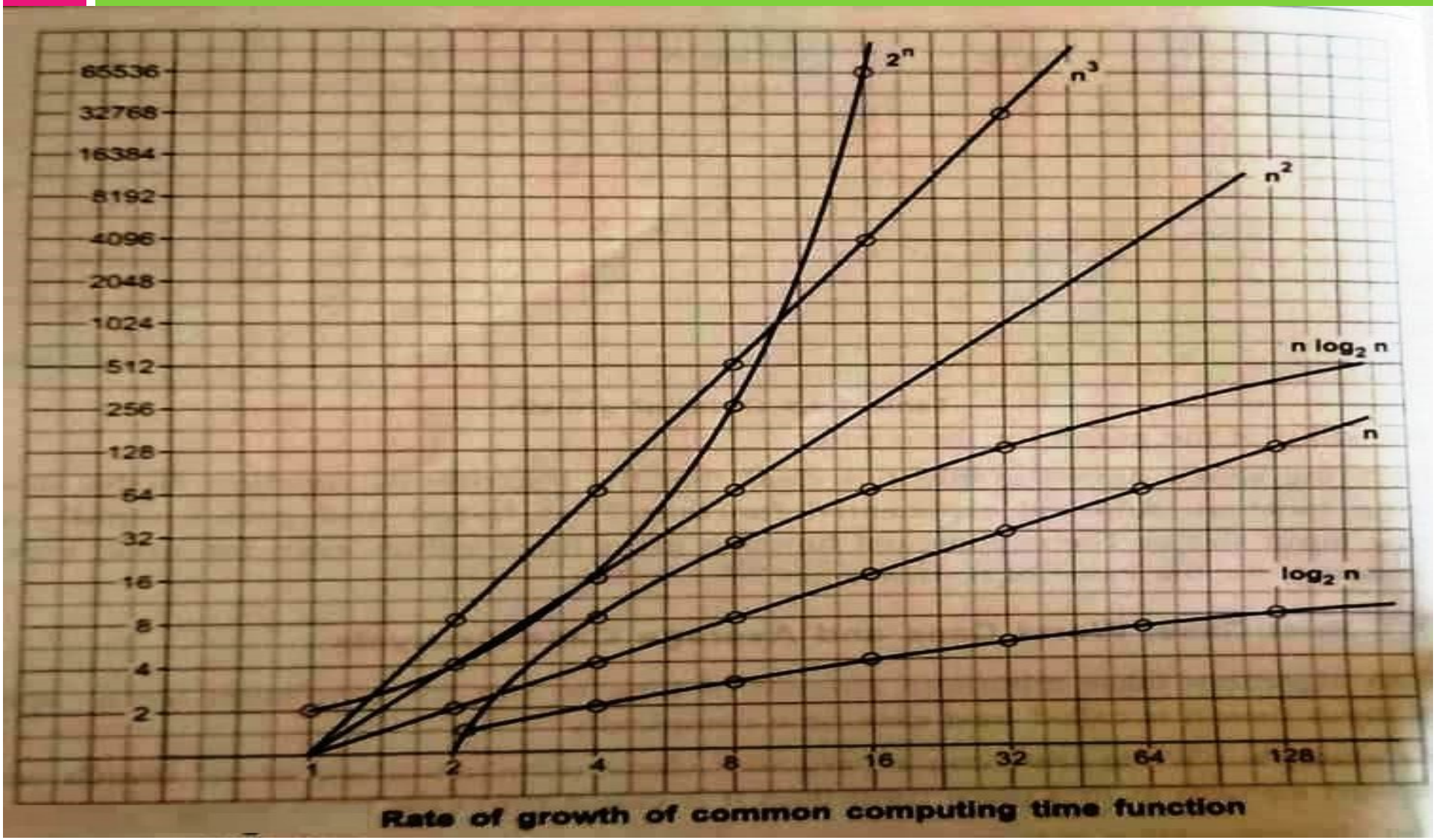
Order of Growth/Performance Classification

n	log n	n log n	n^2	2^n
1	0	0	1	2
2	1	2	4	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65,536
32	5	160	1024	4,294,967,296

Order of growth

Algorithms – Time Complexity

Order of Growth/Performance Classification



Rate of growth of common computing time function

Algorithms

Asymptotic notation- big-o, theta and omega

Cases in complexities:

There are two commonly studied cases of complexity in algorithms:

1. **Best case complexity:** The best-case scenario for an algorithm is the scenario in which the algorithm performs the **minimum amount of work** (e.g. takes the shortest amount of **time**, uses the least amount of **memory**, etc.).
2. **Worst case complexity:** The worst-case scenario for an algorithm is the scenario in which the algorithm performs the maximum amount of work (e.g. takes the longest amount of time, uses the most amount of memory, etc.).
3. **Average case complexity:** The Average case scenario for an algorithm is the scenario in which the algorithm performs between the minimum and the maximum amount of work.

In analyzing the complexity of an algorithm, it is often more informative to study the worst-case scenario, as this gives a guaranteed upper bound on the performance of the algorithm. Best-case scenario analysis is sometimes performed, but is generally less important as it provides a lower bound that is often trivial to achieve.

Algorithms

Asymptotic notation- big-o, theta and omega

- The main idea of asymptotic analysis is to have a **measure of the efficiency of algorithms** that **don't depend on machine-specific constants** and don't require algorithms to be implemented and time taken by programs to be compared.
- **Asymptotic Notations:** are **mathematical tools** used to **analyze** the **performance** of algorithms by **understanding** how their **efficiency changes** as **the input size grows.**
- These notations provide a concise **way to express the behavior** of an algorithm's time or space complexity as the input size approaches infinity.
- Rather than comparing algorithms directly, asymptotic analysis focuses on understanding the relative growth rates of algorithms' complexities.

Algorithms

Asymptotic notation- big-o, theta and omega

- Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.
- By using asymptotic notations, such as Big O, Big Omega, and Big Theta, we can categorize algorithms based on their worst-case, best-case, or average-case time or space complexities, providing valuable insights into their efficiency.
- There are mainly **three asymptotic notations:**
  - **Big-O Notation (O-notation)**
  - **Omega Notation (Ω-notation)**
  - **Theta Notation (Θ-notation)**

Algorithms

Asymptotic notation: Big-O

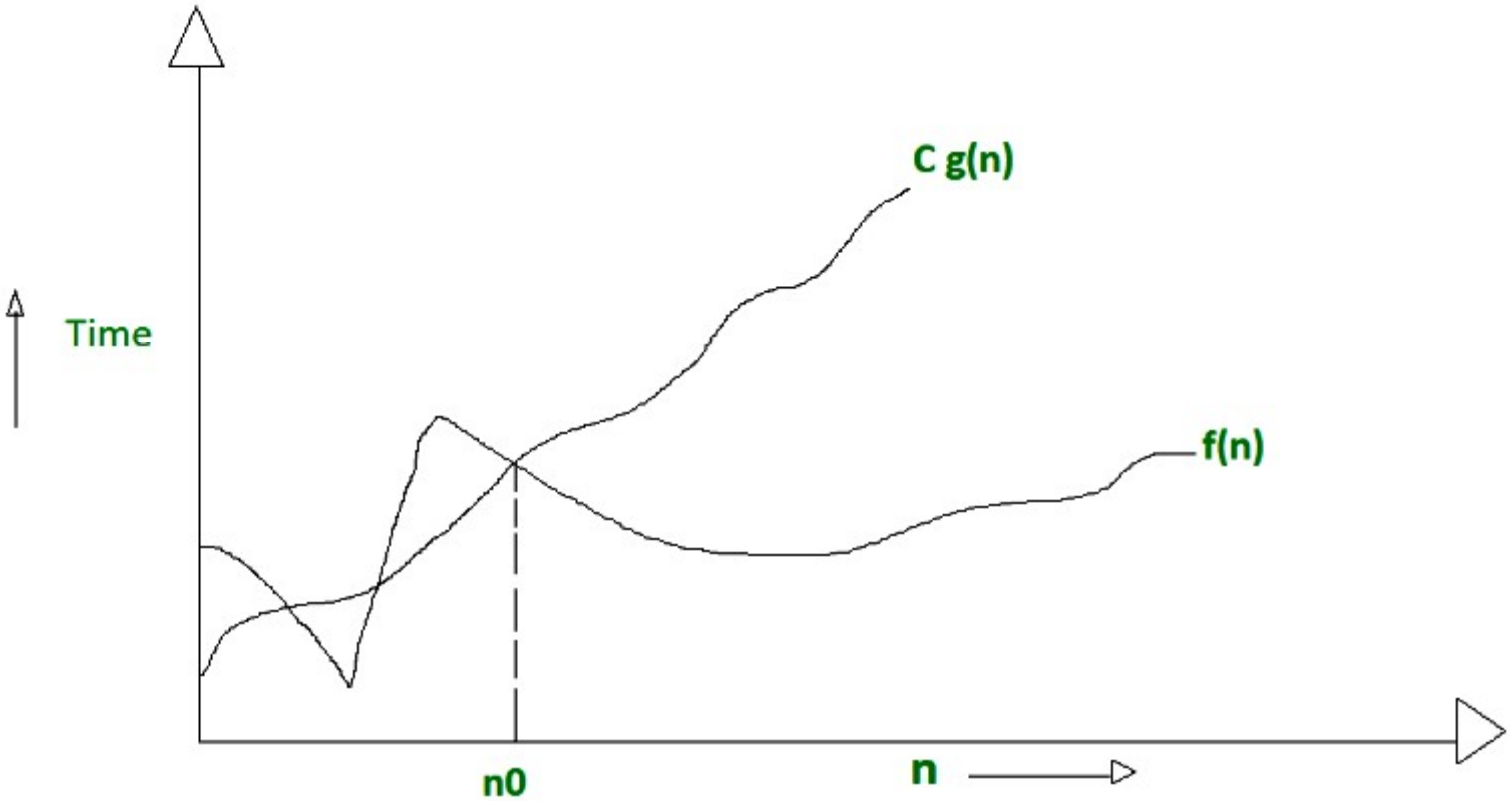
1) Big-O Notation (O-notation):

- It represents the upper bound of the running time of an algorithm i.e. longest amount of time possible, the maximum time required by an algorithm to complete the execution statements. Therefore, it gives the worst-case complexity of an algorithm.
- **Big O notation** is used to describe the **asymptotic upper bound**.
- If f(n) describes the running time of an algorithm, f(n) is O(g(n)) if there exist a positive constant C and n0 such that,

$$0 \leq f(n) \leq c.g(n) \text{ for all } n \geq n0$$

Algorithms

Asymptotic notation: Big-O



$$f(n) = O(g(n))$$

Algorithms

Asymptotic notation: Omega

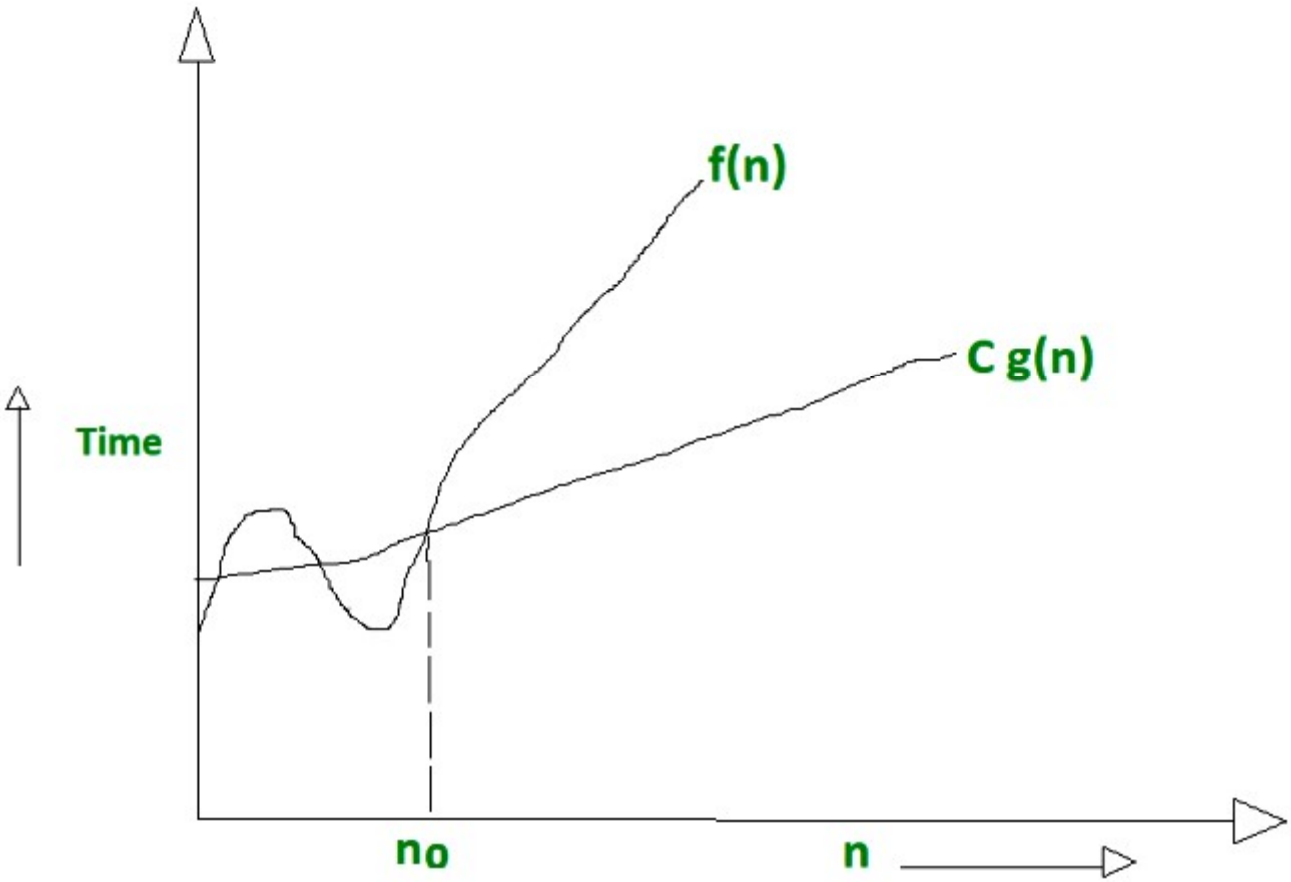
2) Omega Notation (Ω-Notation):

- Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.
- The execution time serves as **a lower bound** on the algorithm's time complexity.
- It is defined as the condition that allows an algorithm to complete statement execution in the **shortest amount** of time.
- Let g and f be the non negative functions from the set of natural numbers to itself. The function f is said to be $\Omega(g)$, if there is a constant $c > 0$ and a natural number n0 such that

$$0 \leq \quad f(n) >= c.g(n) \text{ for all } n \geq n0$$

Algorithms

Asymptotic notation: Omega



$$f(n) = \Omega(g(n))$$

Algorithms

Asymptotic notation: Theta

3) Theta Notation (Θ-Notation):

- Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the **average-case** complexity of an algorithm.
- Let g and f be the function from the set of natural numbers to itself. The function f is said to be Θ(g), if there are constants c1, c2 > 0 and a natural number n0 such that

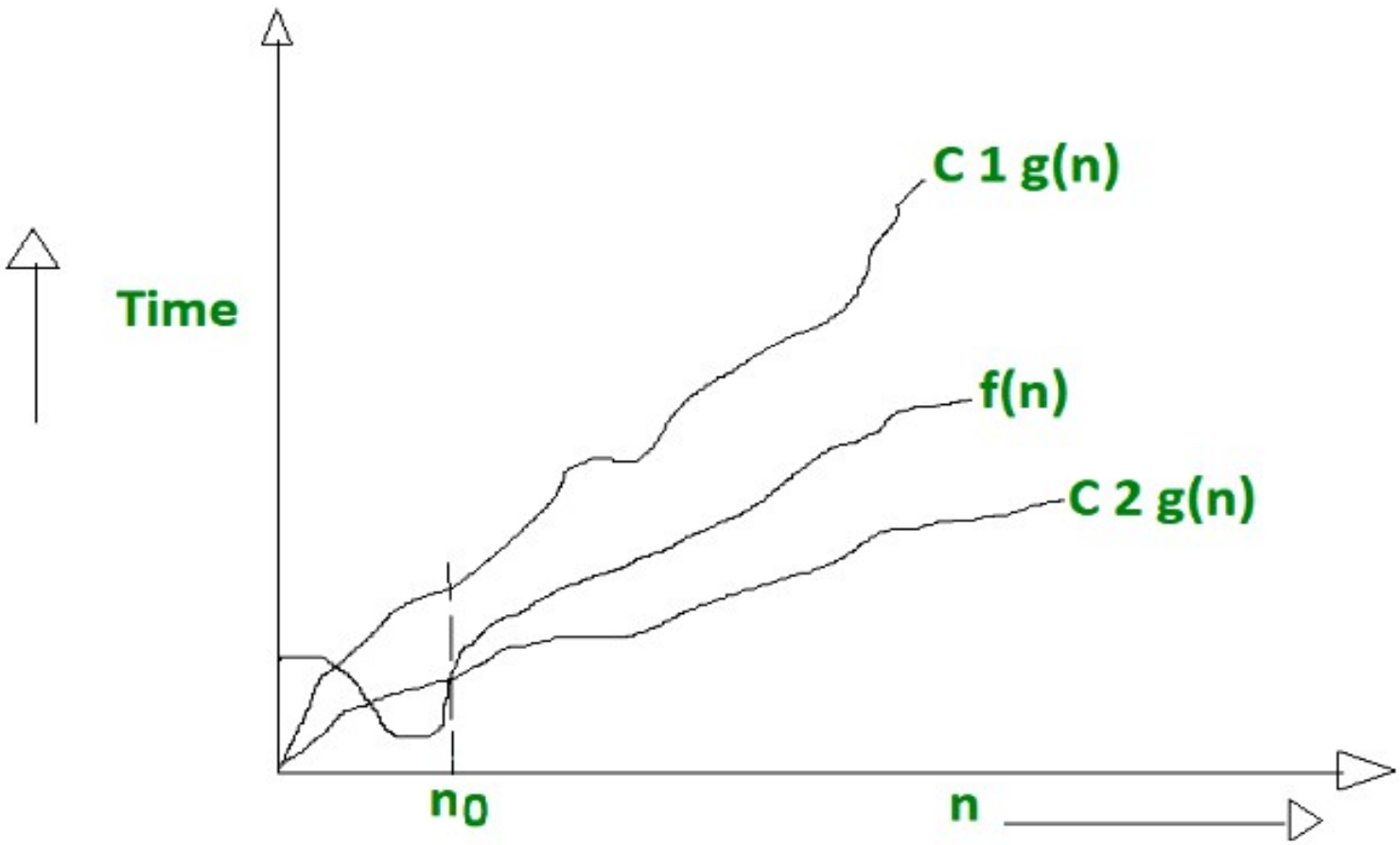
$$c1.\ g(n) \leq f(n) \leq c2.g(n) \text{ for all } n \geq n0$$

Mathematically,

- **f(n)** is said to be **Θ(g(n))** if **f(n)** is **O(g(n))** and **f(n)** is **Ω(g(n)).**
- 0 <= f(n) <= C1g(n) for n >= n0
  0 <= C2g(n) <= f(n) for n >= n0

Algorithms

Asymptotic notation: Theta



$$f(n) = \Theta(g(n))$$

S.No.	Big O	Big Omega (Ω)	Big Theta (Θ)
1.	It is like (<=) rate of growth of an algorithm is less than or equal to a specific value.	It is like (>=) rate of growth is greater than or equal to a specified value.	It is like (==) meaning the rate of growth is equal to a specified value.
2.	The upper bound of algorithm is represented by Big O notation. Only the above function is bounded by Big O. Asymptotic upper bound is given by Big O notation.	The algorithm's lower bound is represented by Omega notation. The asymptotic lower bound is given by Omega notation.	The bounding of function from above and below is represented by theta notation. The exact asymptotic behavior is done by this theta notation.
3.	Big O – Upper Bound	Big Omega (Ω) – Lower Bound	Big Theta (Θ) – Tight Bound
4.	It is define as upper bound and upper bound on an algorithm is the most amount of time required (the worst case performance).	It is define as lower bound and lower bound on an algorithm is the least amount of time required (the most efficient way possible, in other words best case).	It is define as tightest bound and tightest bound is the best of all the worst case times that the algorithm can take.
5.	Mathematically: Big Oh is 0 <= f(n) <= Cg(n) for all n >= n0	Mathematically: Big Omega is 0 <= Cg(n) <= f(n) for all n >= n0	Mathematically – Big Theta is 0 <= C2g(n) <= f(n) <= C1g(n) for n >= n0

Algorithms

Step Count Method

Finding complexity using **Step** / **Frequency Count method**

Comments:

- Comments are used for giving extra meaning to the program. They are not executed during the execution. Comments are ignored during execution.
- Therefore the number of times that a comment executes is **0**.

Conditional statements:

- Conditional statements check the condition and if the condition is correct then the conditional subpart will be executed. So the execution of conditional statements happens only once. The compiler will execute the conditional statements to check whether the condition is correct or not so it will be executed one time.
- In **if-else** statements the if statement is executed one time but the else statement will be executed **zero** or **one** time because if the "if" statement is executed then the else statement will not execute.
- In **switch** case statements the starting switch(condition) statement will be executed one time but the inner case statements will execute if none of the previous case statements are executed.
- In **nested if and if else** ladder statements also initial if statement is executed at least once but inner statements will be executed based on the previous statements' execution

Algorithms

Step Count Method

Loop statements:

- Loop statements are iterative statements. They are executed one or more times based on a given condition.
- A typical **for(i = 0; i ≤ n; i++)** statement will be executed "**n+1**" times for the first n times the condition is satisfied and the inner loop will be executed and for the $(n+1)^{th}$ time the condition is failed and the loop terminates.
- **While:** The statement is executed until the given condition is satisfied.
- **Do while:** The statement will repeat until the given condition is satisfied. The do-while statement will execute **at least once** because for the first time it will not check the condition.

Functions:

- Functions are executed based on the number of times they get called. If they get called **n** times they will be executed **n** times. If they are **not called** at least once then they will not be executed.
- Other statements like **BEGIN, END** and **goto** statements will be executed **one time**

Algorithms

Step Count Method

Analysis of Linear Search algorithm

```
Linearsearch(arr, n, key)
    {
        i = 0;
        for(i = 0; i < n; i++)
        {
            if(arr[i] == key)
            {
                printf("Found");
            }
    }
```

Where, **i = 0**, is an initialization statement and takes O(1) times.

- **for(i = 0;i < n ; i++)**, is a loop and it takes O(n+1) times .
- **if(arr[i] == key)**, is a conditional statement and takes O(1) times.
- **printf("Found")**, is a function and that takes O(0)/O(1) times.
- So, Total Number of times it is executed is **n + 4** times, ignore lower exponents in time complexity total time became **O(n)**.
- **Time complexity:** O(n). **Auxiliary Space:** O(1)

Algorithms

Step Count Method


```
int sum(int n)
{
        int partialSum;

1       partialSum=0;                    1
2       for (int i=1;i<=n;i++)           2N+2 (=1 + (N+1) + N)
3          partialSum += i*i*i;          4N
4       return partialSum;               1
}
```

- Lines 1 and 4 count for one unit each.
- Line 2: (1 for initialization, N+1 for all tests, N for all increments) total 2N + 2.
- Line 3: executed N times, each time four units. total time units: 6N + 4. (⇒ O(N) or O(n))